Formal Verification of Privacy Pass

Kristiana Ivanova* Daniel Gardham† Stephan Wesemeyer

Surrey Centre for Cyber Security
University of Surrey
Guildford, UK
*k.ivanova@surrey.ac.uk, †d.gardham@surrey.ac.uk

***Abstract*—CAPTCHA is a ubiquitous challenge-response system for preventing spam (typically bots) on the internet. Requiring users to solve visual challenges, its design is inherently cumbersome, and can unfairly punish those using low-reputation IP addresses, such as anonymous services, e.g. TOR.**

To minimise the frequency with which a user must solve CAPTCHAs, Privacy Pass (Davidson et al. PETS 2018) allows users to collect and spend anonymous tokens instead of solving challenges. Despite 400,000 reported monthly users and standardisation efforts by the IETF, it has not been subject of symbolic verification, which has been proven to be a valuable tool in security analysis.

In this paper, we perform the first analysis of Privacy Pass using formal verification tools, and verify standard security properties hold in the symbolic model. Motivated by concerns of Davidson et al. and the IETF contributors, we also explore a stronger attack model, where key leakage uncovers a potential token forgery. We present a new protocol, Privacy Pass Plus, in which we show the attack fails in the symbolic model and give new computational proofs to show our scheme also maintains cryptographic security. Moreover, our work also highlights the complementary nature of analysing protocols in both symbolic and computational models.

***Index Terms*—Formal Verification, Symbolic Analysis, Privacy**

I. Introduction

Imperva's bad bot report [1] indicated that in 2024, bots generated 51% of the total internet traffic, surpassing human-generated traffic for the first time in a decade, with 37% being classified as bots with malicious tasks (i.e., bad bots). As reported in DataDome's Global Bot Security Report 2024 [2], bad bots pose a serious threat to the web; they can be utilised for distributed denial-of-service (DDoS) attacks, fraud, stealing, or creating accounts, amongst many other attacks.

CAPTCHAs [3] are ubiquitous challenge-response tests preventing spam on the internet, especially against bots. The mechanism works by serving a challenge to requests from IP addresses (sometimes conditioned on those with low reputation), the response to which determines whether the request was made by a human. However, their challenging interactive design plagues users, and due to their reliance on visual design, those with visual impairments can find them difficult to solve [4]. Furthermore, this approach is seemingly incompatible with anonymising technologies such as virtual private networks (VPNs), TOR[1], and Lokinet[2] because these technologies re-route internet traffic. More precisely, honest users may end up requesting a resource from an IP with a low reputation score, due to previous activity by other users. As a result, these users may be asked to complete CAPTCHA checks more often [5], resulting in a negative user experience and ultimately inhibiting the uptake of privacy-preserving web technologies.

Privacy Pass [6] is a cryptographic protocol developed and deployed by Cloudflare that alleviates these challenges by presenting users with several anonymous tokens upon successful completion of a CAPTCHA challenge. The protocol allows users to redeem tokens in subsequent requests to a resource instead of completing a CAPTCHA, thus significantly lowering the frequency in which an honest user is required to solve these challenges. The security properties of Privacy Pass ensure the client cannot generate their own tokens (called *one-more token security*), and the token issuer (TI) should not be able to link two tokens, therefore offering client privacy (called *unlinkability*). The final security property, called *single-signing key security*, assures the client that the token issuer is not performing de-anonymising attacks by tracking tokens in small anonymity groups.

Despite currently going through standardisation (IETF RFC 9578 [7]) and its browser extension having over 400,000 reported users [8], the Privacy Pass protocol has only had limited security analysis. Davidson et al. [6] demonstrate the security properties through cryptographic reductions, and whilst this builds confidence in the cryptographic design of the protocol, it does not capture use of such a protocol in wider environments where it is deployed. Symbolic approaches allow automated reasoning using techniques such as model-checking, resolution, and rewriting. Whilst there are many formal verification tools, for example ProVerif [9], Squirrel [10], and AVISPA [11], the Tamarin Prover [12] is a state-of-the-art model checker that supports both attack finding and proving in the symbolic model. It has been successfully used to analyse a number of real-world protocols such as TLS [13, 14], FIDO Universal Authenticator Framework [15], and 5G (e.g. [16, 17]). Symbolic verification is an important tool to gain confidence in security of systems and protocols, but as yet, Privacy Pass has no known analysis.

Another open problem for Privacy Pass lies with the security property *Single Signing Key* (SSK), which aims to prevent anonymity attacks by the token issuer. Davidson et. al. [6] highlight the importance of enabling key rotation to be implemented by the TI to both account for when a key has exceeded its lifetime as well as to reduce the risk of attacks such as token stockpiling. SSK mitigates this by decreasing the period in which tokens can be redeemed. However, Davidson et al. [18] demonstrate that this reduction directly impacts client anonymity, especially when considering small key cycles of ≈3-4 days. Furthermore, Tyagi et. al. [19] observe that a malicious signing entity could go as far as to have separate

[1]https://www.torproject.org/
[2]https://lokinet.org/

keys for each client, which would completely break unlinkability. As protection against this, the authors suggest clients could use public ledgers or confirm that the public keys are the same via gossip protocols, which would complicate the infrastructure of the protocol. They also point out that replacing the keys in their storage locations is an error-prone process that can lead to key leakage, which potentially could undermine the unforgeability property of the protocol. Ultimately, this leaves the state of key rotation in an unsatisfactory place: more regular rotations decrease user anonymity and the overhead of additional key management becomes burdensome and error-prone; on the other hand, long cycles would allow for significant *undetectable* abuse, in the event of key leakage.

Security Models. As mentioned previously, symbolic approaches abstract away the concrete algebraic properties of cryptography and reason about security using the Dolev–Yao attacker model, which assumes unbounded computational power but restricts the attacker to constructing only symbolic terms from known information. This abstraction simplifies reasoning and has enabled the development of effective automated verification tools. However, the symbolic model does not provide guarantees in the computational sense and as a result, protocols proven secure symbolically may still be vulnerable to attacks that exploit real-world cryptographic properties. Moreover, protocol verification is a subtle art which is prone to human error and missed edge cases. Having two distinct but complementary approaches to verify the security of a protocol increases the confidence in the soundness of the analysis. The idea of reconciling these two approaches to combine automation and scalability with realistic cryptographic guarantees was first proposed by Abadi and Rogaway [20] and since then a number works have exploited this unified approach, including for avionic protocols [21] and in analysis of symmetric encryption [22].

Contributions. This paper provides 3 contributions to the security analysis and development of the Privacy Pass protocol, summarised as follows.

1) We provide the first symbolic formalisation of Privacy Pass and its security properties in the TAMARIN PROVER [12], inspired the computational definitions presented by Davidson *et al*. [6]. Our results show that the protocol meets the one-more token security, single signing key and unlinkability properties. To do this, we also present the first computational formalisation of the SSK property for Privacy Pass.
2) We examine the protocol in a stronger threat model that captures key leakage from the token issuer, and allows the adversary to eavesdrop and inject messages. We find that the current Privacy Pass protocol is insecure, and we present an attack found by our TAMARIN model in which an adversary can create forged tokens using leaked keys.
3) We propose a new protocol, called Privacy Pass Plus, and show that it mitigates the key leakage attack when modelled in the TAMARIN PROVER. To achieve this, we allow the Server to "spot check" token redemptions to ensure validity in the event of unexpected behaviour such as sudden influx of requests. This additional check mitigates against the token issuer's key leakage, and ultimately affords the Server the ability to detect it. If the check fails, then the Server can refuse to serve the request. In turn, this extra assurance means key rotation can have larger time periods, which reduces key management overhead and ultimately allows for larger anonymity groups for the user. Finally, in addition to our symbolic analysis, we also provide cryptographic reductions to show it still satisfies the unlinkability and single signing key properties, and a stronger notion of one-more-token security that captures the TI key leakage.

A. Related Work

One approach to minimising impact of token issuer key compromise is to limit the rate at which tokens can be spent on a per-origin basis. This Privacy Pass variant was proposed by Hendrickson et al. in the IETF *rate-Limited Token Issuance Protocol* draft [23]. This is facilitated by a newly introduced party, called the attestor, which verifies the client's signature and performs additional checks upon receiving the request, such as ensuring the request number is below some agreed threshold. Should this threshold be exceeded, the attestor would reject any requests from or to that client or issuer, respectively, until the values are reset. Furthermore, the attestor is responsible for separating the sensitive information, like the client's identity, from the information needed for the token issuance, needed to protect user privacy from the Token Issuer.

Chu et al. [24] formalize the security of the rate-limiting version of the Privacy Pass protocol using a game-based security model. They focus on two main properties–privacy and unforgeability – adapted for the new attestor party. Unforgeability adopts the security approach of Privacy Pass, and intuitively requires the adversarial client to submit $n+1$ valid tokens after only requesting n. This property is known as one-more-token security. The authors consider both client privacy (which captures the notion of unlinkability from Privacy Pass) and a new property *origin privacy*, which ensures that a malicious attestor and corrupt origins could not link token issuance with redemption, thus a token cannot be linked to the origin for which it was requested. However, ultimately, the security of this protocol does require the introduction of an additional trusted party, which complicates the protocol and increases the attack surface for Privacy Pass, which we avoid in this work.

We also note that Hanff [25] gives a computational analysis of the IETF version of Privacy Pass (RFC 9578), which more generically relies upon Verifiable Oblivious Pseudorandom Functions (VOPRFs). Furthermore, they prove stronger security properties that capture token expiry through redemption context. Whilst offering stronger privacy guarantees than alternative public metadata-based approaches, it does not mitigate the need for a single signing key property, which is not considered in this work. Our analysis covers both computational and symbolic of Privacy Pass from Davidson et al., and in particular, consider the SSK property.

Whilst Privacy Pass itself is lacking formal analysis, symbolic tools have successfully been used to verify many other web-based protocols [26, 27, 28, 29, 30]. In particular, the TAMARIN PROVER has also been used analyse web protocols such as TLS 1.3 [31], which are able to establish authenticated key exchange in both unilateral and mutual cases. They also uncover a potential attack when considering a delayed client authentication mechanism. Furthermore, Hofmeier et al. [32] formally analyse the security properties of the widely deployed *Single Sign-On* (SSO) protocol - *OpenID Connect* (OIDC) which enables users to authenticate themselves to different web services. The security properties proved within this model are centred around unforgeability of an IDtoken. The FIDO2 standard

for online authentication has also been analysed by Schrempp [33], who shows that it meets strong device authentication properties, but fails to guarantee this in the case of user authentication. Moreover, this work considers the human element, demonstrating the versatility of TAMARIN for real-world protocols.

B. Organisation

We introduce the Privacy Pass architecture, its algorithms, and security properties in Section II. In Section III we introduce the TAMARIN PROVER and recast the properties defined in Section II for TAMARIN. We prove that our model for Privacy Pass meets these transposed security definitions. In Section IV we then consider a stronger attack model whereby an adversary can eavesdrop on the channels, inject messages, and also has knowledge of a leaked TI key. We discuss a resulting attack found by this stronger adversarial model. In Section V, we propose a new protocol, Privacy Pass Plus, that is resistant to the attack we found and allows the Server to 'spot check' tokens. We formally show our protocol is secure in TAMARIN and through cryptographic reductions. We end the paper with a brief overview of our findings and conclusions in Section VI.

II. PRIVACY PASS

The Privacy Pass protocol has two main phases - token issuance and token redemption. We define 3 parties and their roles:

User (U) – requests access to web resources; requests Privacy Pass tokens; solves challenges sent by the Server; verifies token issuer's token signatures poofs; redeems Privacy Pass tokens.

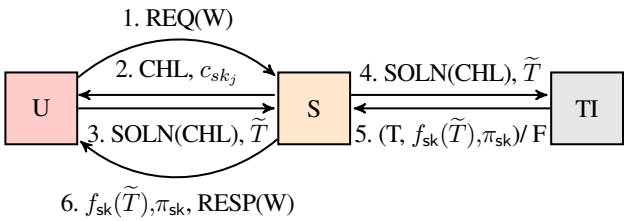
Server (S) – generates and sends challenges to the user; sends web resources to the user; forwards communication between the user and token issuer.

Token Issuer (TI) – verifies the correctness of the user's solution to the Server's challenge; issues the tokens to the user, and verifies the tokens during redemption.

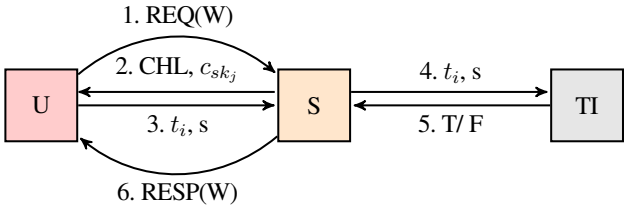
Remark 1. *The formal model of Davidson et al. [6] actually defines four actors: Client, Ticket Issuer, Edge provider, and Origin Website. This is particularly motivated by the CDN use case, however, for generality we have combined the roles of* Edge Provider *and* Origin Website *into* Server *as fundamentally the Origin has no role in the security of the scheme, in particular, it only provides the resource upon successful completion of a CAPTCHA or token redemption.*

A. Privacy Pass Construction

Overview. The token issuance procedure is shown in Figure 1a. It begins with the User U sending a request, REQ(W), for a specific web resource (*W*) from an origin/website hosted by a Server S. In step 2, S sends a challenge (*CHL*) back to the User, who solves it and replies with their answer SOLN(CHL), along with their blinded tokens $\widetilde{T}$. The Server S forwards this to the TI, which verifies a successful solution SOLN(CHL), and signs the blinded tokens, $f_{\mathsf{sk}}(\widetilde{T})$ using its secret key sk and some signing algorithm f, and computes a proof of the signature π_{sk}. These, together with *W*, are then sent back to the User who verifies π_{sk}, performs an unblinding, to produce signed tokens $f_{\mathsf{sk}}(T)$ which are stored in the User's *Privacy Pass* browser extension. To redeem tokens, as depicted in Figure 1b, the Server receives the request REQ(W) and issues a challenge as before, but this time the User replies with a



(a) Issuance Flow



(b) Redemption Flow

Fig. 1: Privacy Pass Protocol Overview

token seed t_i and some token verification data s. The TI checks the validity of s with respect to t_i and returns true or false accordingly. If successful, the Server serves the response RESP(W) to the User.

To build out the algorithms that instantiate the procedures shown in Figure 1, we first define some functions defined over a group $\mathbb{G}$ with generator g of order q. Note that Message Authentication Codes (MAC), Non-interactive Zero-Knowledge Proofs (NIZK), and Commitment schemes (Com) are formally defined, along with their security properties, in Appendix A. H_1, H_2 and H_3 are collision-resistant hash functions. B_U is a stack for the user, and B_s is a stack for the server.

$\mathsf{sKGen}(t,T')$ Takes as input a seed t and token T' and returns $K = H_2(t,X)$ for a hash function $H_2 : \{0,1\}^* \to \mathbb{K}$, where $\mathbb{K}$ is the key space for MAC.

$\mathsf{GenToken}(t)$ Takes as input a seed t and returns a token $T = H_1(t)$ for a hash function $H_1 : \{0,1\}^* \to \mathbb{G}$.

$\mathsf{Blind}(T)$ This algorithm takes as input a token T, samples an integer r from $\mathbb{Z}_q$ and computes $\widetilde{T} = T^r$. It outputs $\{T,r\}$.

$\mathsf{Unblind}(\widetilde{T},r)$ This algorithm takes as input a blinded token $\widetilde{T}$ and randomness r, and returns $T = \widetilde{T}^{1/r}$.

$\mathsf{Sign}(\mathsf{sk},T)$ Sign takes as input a token T and a signing key sk and returns the signed token $f_{\mathsf{sk}}(T) = T^{\mathsf{sk}}$.

$\mathsf{Store}(\cdot)$ A management algorithm that stores a tuple to the stack.

$\mathsf{Pop}()$ Takes no input, this algorithm pops the last tuple in the stack.

Protocol Setup. Define an algorithm Setup that takes as input a security parameter λ and returns the public parameters pp for the scheme and are implicit inputs to all algorithms. In this case, the parameters are the description of the group $\mathbb{G}$. Also define a key generation algorithm KeyGen that takes as input the public parameters pp and returns a secret key sk. For this construction, this is simply $\mathsf{sk} \leftarrow\$ \mathbb{Z}_q$. Finally, using a commitment scheme, it outputs a commitment c_{sk} to sk.

Token Issuance. We now formally describe the concrete construction for token issuance, depicted in Figure 2, under the assumption that the CAPTCHA is solved correctly. We break it down into the following phases to capture the interactive nature: $\mathsf{U\text{-}Issue}_1$, $\mathsf{TI\text{-}Issue}$, $\mathsf{U\text{-}Issue}_2$– this can be mapped to the figure where each

phase is separated by some information flow depicted by an arrow. To begin, the TI generates a secret key $\mathsf{sk} \leftarrow\$ \mathbb{Z}_q$ and creates a commitment $c\ (= g^{\mathsf{sk}})$, which it broadcasts publicly. To begin token issuance, the User samples a random seed $t \leftarrow\$ \mathbb{Z}_q$ and uses this to generate a token T from the GenToken algorithm, which is then blinded (using the Blind algorithm) to create a blinded token $\widetilde{T}$ and its randomisation $r \in \mathbb{Z}_q$, $\widetilde{T}$ is sent to the token issuer. The TI invokes the signing algorithm with its key sk to create the signed blinded token $f_{\mathsf{sk}}(\widetilde{T})$, which is returned to the client with a *Discrete-log Equality* (DLEQ) proof π_{sk} that sk is the same as one committed to in c (c.f. Figure 13 in Appendix A). The user checks the proof holds by executing NIZK.Verify, and aborts otherwise. If it passes, then the user unblinds the token using Unblind, and stores the output, along with the seed t.

Token Redemption. We formally describe the redemption phase in Figure 3. We break it down into the following phases to capture the interactive nature: U-Redeem, TI-Redeem. As before, this can be seen in the figure, where each phase is separated by an information flow along an arrow. Intuitively, the unblinded tokens can be spent by proving to the TI that the User has knowledge of a signed token. To this end, the client pops a signed token $f_{\mathsf{sk}}(T)$ and its seed t from the stack, uses these to derive a MAC key k, which is then combined with the request data R[3] to produce a MAC s. The client sends s and the seed t to the TI for redemption. The TI performs a check to prevent double-spending of a token; in particular, it checks if the seed t is saved in its storage, and rejects if so. Else, since the key k is deterministically derived, the TI can recreate it from both t and sk by re-deriving T and signing it once more. It can also compute a MAC s', and verify the token is valid if $s \stackrel{?}{=} s'$. In which case, the TI stores the seed t by adding it to the stack B_S, thus marking it as spent. Finally, the TI sends the Server a response that the client is verified, and in turn, S sends the requested web resource back to the client.

B. Correctness and Security Properties of Privacy Pass

In this section, we formalise the game-based security experiments of Privacy Pass based on the definitions presented by Davidson et al. [6], which will serve as inspiration for defining security in our Tamarin model.

In our definitions, we use $\varepsilon(\lambda)$ to represent a negligible function in the security parameter λ. Throughout all experiments, the request data R is some fixed value, and so we suppress this in our modelling. Furthermore, the setup algorithm public parameters pp which are assumed to be implicit inputs to all algorithms.

1) Oracles: Our security experiments make use of the oracles:

$\mathcal{O}_{\mathsf{U\text{-}Issue}_1}$ This oracle takes no input and initiates the issuance of tokens. It samples a seed t and runs the algorithm GenToken, then outputs blinded tokens $\widetilde{T}$. It adds the tuple $(\mathsf{sid}, t, r, \widetilde{T})$ to the list IssuedTokens, where sid is a session ID incremented from 0, and r is the blinding randomness.

$\mathcal{O}_{\mathsf{TI\text{-}Issue}}$ For a secret key sk fixed by the experiment, this oracle takes as input $\widetilde{T}$ and returns signed blinded tokens $f_{\mathsf{sk}}(\widetilde{T})$.

$\mathcal{O}_{\mathsf{U\text{-}Issue}_2}$ This oracle takes as input $(\mathsf{sid}, f_{\mathsf{sk}}(\widetilde{T}))$, unblinds $f_{\mathsf{sk}}(\widetilde{T})$ and returns the signed tokens $f_{\mathsf{sk}}(T)$ and the corresponding seed t (matched from the list IssuedTokens on sid). If $(\mathsf{sid}, \cdot, \cdot, \cdot) \notin \mathsf{IssuedTokens}$ then it returns $\perp$.

[3]This can be viewed as the request host header concatenated with the HTTP path.

$\mathcal{O}_{\mathsf{U\text{-}Redeem}}$ This takes as input the signed tokens and session ID $(\mathsf{sid}, f_{\mathsf{sk}}(T))$. It locates $(\mathsf{sid}, \cdot, \cdot, \cdot) \in \mathsf{IssuedTokens}$ and extracts the seed t and computes verification data s. It checks a sidList list for sid for every token redeemed to ensure it has not previously been spent. If sid is already in sidList, it aborts, else it outputs (t,s) and adds sid to the sidList to mark the token as spent.

$\mathcal{O}_{\mathsf{TI\text{-}Redeem}}$ This oracle takes as input the seed t and computes verification data s. It checks if s is valid with respect to t. Further, it checks if $t \notin \mathsf{SpentTokens}$, in which case it adds t and returns 1, else it returns 0.

$\mathcal{O}_{\mathsf{Chal}}$ This challenge oracle is used in the unlinkability experiment. It takes input two signed blinded tokens $f_{\mathsf{sk}}(\widetilde{T}_0)$ and $f_{\mathsf{sk}}(\widetilde{T}_1)$ (and resp. sid), and then executes and returns $\mathcal{O}_{\mathsf{U\text{-}Redeem}}(\mathsf{sid}_b, f_{\mathsf{sk}}(T_b))$ for both $b=0$ and $b=1$. If the oracle returns $\perp$ for either $b=0$ or $b=1$, then it simply returns $\perp$. Else it returns (t_b, s_b), according to the bit b defined by the experiment.

Correctness. To ensure that a protocol, implementing the issuance and redemption phases, operates consistently and as intended, Davidson et al. [6] provide a generic correctness definition. Correctness ensures that if the protocol is executed in a secure environment without interference from adversaries, the output of the redemption phase is valid. The formal definition is given in Definition 1.

Definition 1. *We say that the protocol* Π *is correct if the probability that a token generated in the* Π.IssuePhase *can successfully be redeemed in the* Π.RedeemPhase *is negligibly close to 1, i.e.*

$$\Pr\left[1 \leftarrow \Pi.\mathsf{RedeemPhase}(t, f_{sk}(T)) \;\middle|\; \begin{array}{c} t \xleftarrow{\$} \mathbb{Z}_q, \\ f_{sk}(T) \leftarrow \Pi.\mathsf{IssuePhase}(t) \end{array}\right] > 1 - \varepsilon(\lambda)$$

One-More-Token Security. Intuitively, one-more-token security provides the unforgeability property for the protocol. It ensures that with honest setup and key generation, a malicious User cannot successfully redeem $n+1$ tokens after only receiving n tokens from oracle $\mathcal{O}_{\mathsf{TI\text{-}Issue}}$. This is captured in our experiment by requiring the adversary to generate "one-more" valid token than was issued by the issue oracle. This is captured by ensuring $n+1$ booleans res_i for $i \in [1, n+1]$ all return true. We highlight that the redemption algorithms on line 5 are implicitly stateful, and thus capture an adversary's attempt to double-spend a token. This matches the stronger notion for unforgeability as first defined by Lehmann et al. [25]. The game is formally defined in Figure 4.

Definition 2. *We say that the protocol has one-more-token security if the advantage of the PPT adversary* $\mathcal{A}$ *in the experiment* $\mathsf{Exp}^{\mathsf{one\text{-}more}}_{\mathcal{A},\Pi}(\lambda)$ *in Figure 4 is negligible in the security parameter* λ*, i.e.*

$$\mathsf{Adv}^{\mathsf{one\text{-}more}}_{\mathcal{A},\Pi}(\lambda) := \left|\Pr\left[\mathsf{Exp}^{\mathsf{one\text{-}more}}_{\mathcal{A},\Pi}(\lambda) = 1\right]\right| \leqslant \varepsilon(\lambda)$$

Unlinkability. This property provides the client anonymity guarantees. Davidson et al. [6] formally define the unlinkability of the token issuance and redemption phases in the Privacy Pass protocol as the view of the token signing entity (the TI) being indistinguishable from an element of $\mathbb{G}$ chosen uniformly at random. We opt instead to generalise the definition away from the underlying mathematical structures, and instead measure the advantage an adversary has in distinguishing between two users redeeming their tokens. The two-stage stateful adversary is also

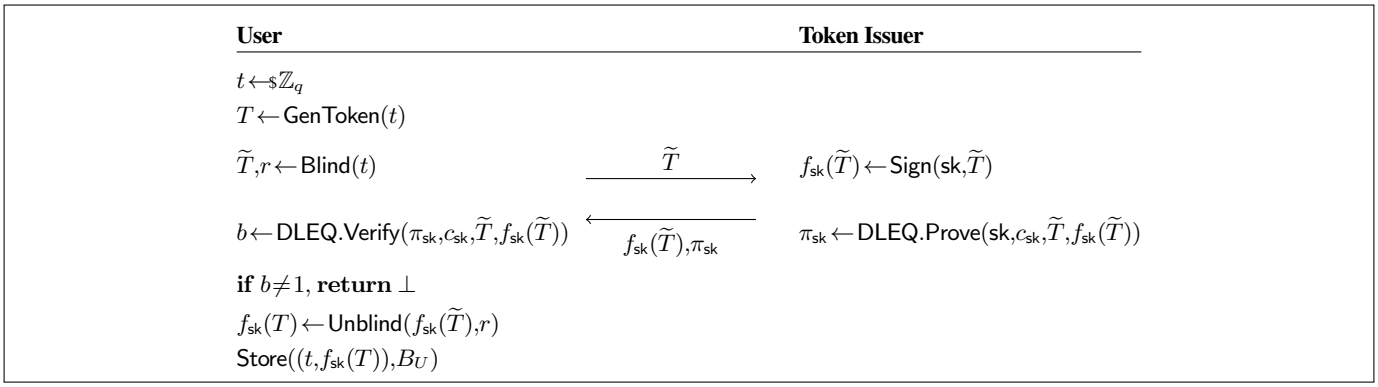


Fig. 2: Privacy Pass token issuance

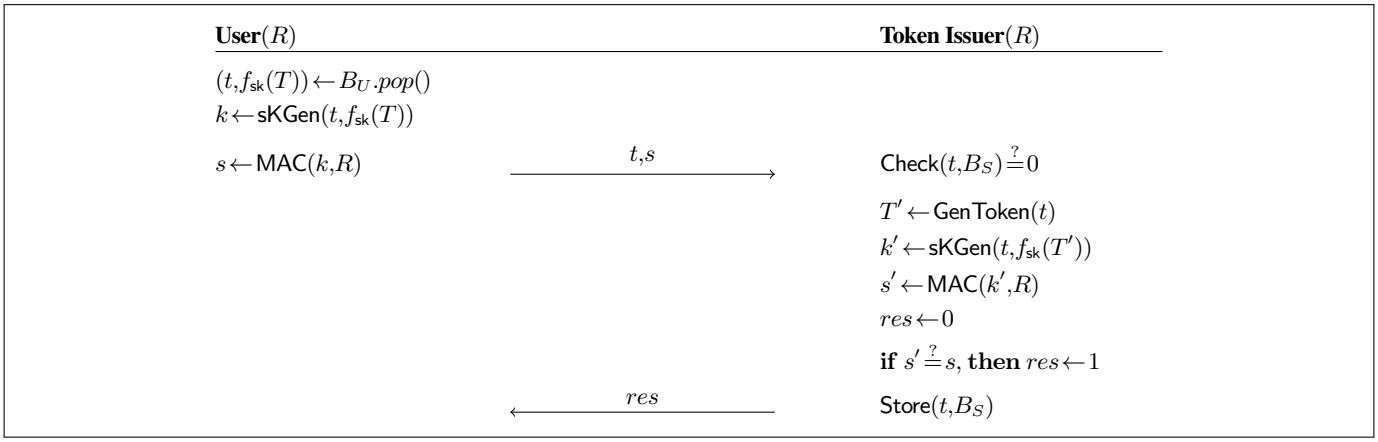


Fig. 3: Privacy Pass token redemption

$\mathsf{Exp}^{\mathsf{one\text{-}more}}_{\mathcal{A},\Pi}(\lambda)$

1: $pp \leftarrow \mathsf{Setup}(\lambda)$
2: $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KeyGen}(\lambda)$
3: $\{\mathsf{tok}_i\}_{i=1}^{n+1} \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{TI\text{-}Redeem}}, \mathcal{O}_{\mathsf{TI\text{-}Issue}}}$
4: **for** $i \in [1, n+1]$
5: $\quad res_i \leftarrow \mathsf{TI\text{-}Redeem}(\mathsf{sk}, \mathsf{U\text{-}Redeem}(\mathsf{tok}_i))$
6: **return** $\bigwedge_{i=1}^{n+1} res_i$

Fig. 4: One-more Token experiment

$\mathsf{Exp}^{\mathsf{unlink}\text{-}b}_{\mathcal{A},\Pi}(\lambda)$

1: $pp \leftarrow \mathsf{Setup}(\lambda)$
2: $\mathsf{bt}_0 \leftarrow \mathsf{U\text{-}Issue}_1()$
3: $\mathsf{bt}_1 \leftarrow \mathsf{U\text{-}Issue}_1()$
4: $(\mathsf{sbt}_0, \mathsf{sbt}_1) \leftarrow \mathcal{A}_0(\mathsf{bt}_0, \mathsf{bt}_1)$
5: $\mathsf{tok}_0 \leftarrow \mathsf{U\text{-}Issue}_2(\mathsf{sbt}_0)$
6: $\mathsf{tok}_1 \leftarrow \mathsf{U\text{-}Issue}_2(\mathsf{sbt}_1)$
7: **abort if** $\mathsf{tok}_0 = \perp \vee \mathsf{tok}_1 = \perp$
8: $b' \leftarrow \mathcal{A}_1^{\mathcal{O}_{\mathsf{U\text{-}Redeem}}, \mathcal{O}_{\mathsf{U\text{-}Issue}_1}, \mathcal{O}_{\mathsf{U\text{-}Issue}_2}, \mathcal{O}_{\mathsf{Chal}}}$
9: **return** $b' \stackrel{?}{=} b$

Fig. 5: Unlinkability experiment

equipped with a blinded token and is challenged to identify if the challenge oracle is returning tokens from the blinded token ($b = 0$), or randomly generated tokens ($b = 1$). It may query any oracle up to q times where q is bounded by an arbitrary polynomial in the security parameter. A negligible advantage implies that the TI's view in the two phases are independent, and we give the formal definition in Definition 3, and present the game in in Figure 5.

Definition 3. *We say that the protocol is unlinkable if the advantage of the PPT adversary $\mathcal{A}$ in distinguishing the experiment* $\mathsf{Exp}^{\mathsf{unlink}\text{-}0}_{\mathcal{A},\Pi}(\lambda)$ *from* $\mathsf{Exp}^{\mathsf{unlink}\text{-}1}_{\mathcal{A},\Pi}(\lambda)$ *defined in Figure 5 is negligibly small (in λ), i.e.*

$$\mathsf{Adv}^{\mathsf{unlink}}_{\mathcal{A},\Pi}(\lambda) := \left| \Pr\left[\mathsf{Exp}^{\mathsf{unlink}\text{-}0}_{\mathcal{A},\Pi}(\lambda) = 1\right] - \Pr\left[\mathsf{Exp}^{\mathsf{unlink}\text{-}1}_{\mathcal{A},\Pi}(\lambda) = 1\right] \right| \leqslant \varepsilon(\lambda)$$

Single Signing Key. We now state the first formal definition for single signing key. This property guarantees user anonymity against a malicious token issuer by preventing tracking of users through small anonymity groups. Intuitively, an adversary wins if it is able to produce a token that is accepted by the user, but is not verifiable against a specified and committed key, capturing a deanonymising attack where a TI uses different keys for different users. In the experiment,

the adversary outputs a token tok^* and a key sk. The experiment aborts if tok^* is not accepted, and otherwise computes a token tok by executing the issuance algorithms. The adversary wins if the tokens tok and tok^* are different, yet tok^* is accepted by the NIZK proof verification of the user. Our definition is inspired specifically by verifiable oblivious PRFs such as Albrecht and Gur [34] and Albrecht et al. [35]. The formal game-based definition is given in Definition 4.

Definition 4. *We say that the protocol has the single signing key property if the advantage of the PPT adversary $\mathcal{A}$ in the experiment $\mathsf{Exp}^{\mathsf{single\text{-}key}}_{\mathcal{A},\Pi}(\lambda)$ defined in Figure 6 is negligible in λ, i.e.*

$$\mathsf{Adv}^{\mathsf{single\text{-}key}}_{\mathcal{A},\Pi}(\lambda) := \left|\Pr\left[\mathsf{Exp}^{\mathsf{single\text{-}key}}_{\mathcal{A},\Pi}(\lambda)=1\right]\right| \leqslant \varepsilon(\lambda)$$

Cryptographic Security Analysis of Privacy Pass. Davidson et al. [6] show that the construction for Privacy Pass satisfies correctness and the security definitions Definitions 2 to 4 (albeit the informal definitions) if El Gamal is secure against one-more decryption attacks (in the ROM), if the zero-knowledge proof system NIZK is sound and is zero-knowledge, and the commitment scheme Com has the binding property. For completeness we recall these proofs in Appendix D, adapted to our formal and generalised definitions.

III. Modelling Privacy Pass in Tamarin

In this section, we discuss the methodology, models and security properties of Privacy Pass in Tamarin.

A. Methodology

The Tamarin Prover [12] is a formal verification tool for symbolic modelling, supporting unbounded verification, mutable global state, and user-defined equational theories. Protocols are modelled using multiset rewriting rules, where states and transitions are explicit, and properties are expressed in first-order logic. Tamarin offers automated verification as well as an interactive mode for manual proofs. It can provide proofs or counterexamples, but may fail to terminate due to undecidability.

Tamarin models communication under the Dolev-Yao (DY) adversary model [36], where the adversary controls the network but is limited by perfect cryptography. In this work, we refer to this attack model as D. The adversary `A` can manipulate messages and apply known functions, but can only decrypt or encrypt with the correct key. Cryptographic properties are modelled using equations, enabling the adversary to derive or generate messages.

The protocol's global state is a multiset of facts, representing participants' local states, adversary knowledge, and network messages. Key facts include `In` (message received), `Out` (message sent), `Fr` (fresh value), and `K` (adversary's knowledge). Rules of the form `premise -- action -> conclusion` define protocol actions, consuming and producing facts to update the global state. Executing these rules generates a trace of the protocol runs.

Tamarin supports a number of heuristics to efficiently guide the verification process by reordering the ranking of the open proof goals. When the default heuristics are insufficient or unsuitable for specific proofs, users can build custom proof *oracles* that override the standard heuristic and enable tailored proof strategies.

Security properties are defined as `lemmas` in first-order logic, checked across all or one protocol executions. Advanced properties like unlinkability use observational equivalence by including the `diff` operator and require Tamarin to distinguish between two instances of the same protocol differing in certain terms.

$\mathsf{Exp}^{\mathsf{single\text{-}key}}_{\mathcal{A},\Pi}(\lambda)$

1: $pp \leftarrow \mathsf{Setup}(\lambda)$
2: $\mathsf{bt} \leftarrow \mathsf{U\text{-}Issue}_1()$
3: $(\mathsf{sbt}, \mathsf{sk}, \mathsf{pk}) \leftarrow \mathcal{A}(\mathsf{bt})^{\mathcal{O}_{\mathsf{U\text{-}Issue}_1}, \mathcal{O}_{\mathsf{U\text{-}Issue}_2}}$
4: $\mathsf{tok}^* \leftarrow \mathsf{U\text{-}Issue}_2(\mathsf{pre}, \mathsf{sbt}, \mathsf{pk})$
5: **if** $\mathsf{tok}^* = \bot$, **abort**
6: $\mathsf{tok} \leftarrow \mathsf{TI\text{-}Issue}(\mathsf{sk}, \mathsf{pre})$
7: $b \leftarrow (\mathsf{tok}^* \stackrel{?}{=} \mathsf{tok})$
8: **return** $\bar{b}$

Fig. 6: Single Signing Key experiment

B. Architecture and Modelling Choices

We now describe the model architecture used to faithfully represent Privacy Pass. Firstly, our models only allow for one server and one TI to be instantiated in a single instance of Privacy Pass. This change does not affect the security and functionality of the models since, as described in [37], every server has a specific trusted TI. On the other hand, user actors will not be limited to a single instance, as multiple users would be expected to request resources from the same origin. Indeed, this is the motivation for some of the security properties, such as single signing key and unlinkability.

Since the protocol is implemented over HTTPS in practice, the Tamarin models make use of a secure channel to ensure confidentiality, integrity, and authenticity of the messages sent between the actors. Furthermore, we limit the number of tokens a user can create to 3, as according to the small-scope hypothesis [38], most model faults can be found using a smaller number of instances, in this case – tokens. Therefore, this approach would allow the models to function as specified in Davidson et al. [6] and have a few tokens per user while also ensuring that Tamarin can auto-prove the security properties and (most likely) discover any potential attacks.

Finally, the checks of the user's challenge solution will always return `true` and the double-spend check will always return `false`, to remove trivialities from the models.

a) Modelling Privacy Pass: Our Tamarin models follow the Privacy Pass definition as outlined in Figure 1 with the functions' underlying computations and names adhering to concrete instantiation described in Figures 2 and 3. We use three Tamarin models – one to represent the full Privacy Pass protocol (PP), its correctness, one-more-token and SSK security properties, and two simplified protocols ($\mathsf{PP}_{2.1}$, $\mathsf{PP}_{2.2}$) to prove that an attacker cannot inject or manipulate tokens successfully and to prove unlinkability, respectively. Looking ahead, Table I gives a short description of the Tamarin Privacy Pass models that will be detailed in subsequent sections and respective security properties (formally defined in Section III-C).

Since Davidson et al. [6] do not detail how the request binding data (Section II-A) is computed, we model it as a generic function that takes 3 parameters – the User and TI entities (said to share knowledge of R) and the request for which the token is redeemed.

All models PP, $\mathsf{PP}_{2.1}$, and $\mathsf{PP}_{2.2}$ establish the 3 actors – `Ti`, `Srv` and `Usr` and define the aforementioned architectural

TABLE I: List of the TAMARIN models.

Model	Description	Properties verified
PP	Full Privacy Pass representation	Correctness (*C*) One-more-token security (*SP1*) Single signing key (*SP3*)
$PP_{2.1}$	PP simplified model, insecure communication between the Server and User	Correctness (*C*) Adversary's inability to inject and manipulate tokens (*A2.1-A2.4*)
$PP_{2.2}$	Identical construction to $PP_{2.1}$ with a secure communication between the User and Server to avoid non-termination	Correctness (*C*) Unlinkability (*SP2*)

restrictions. They also describe the equational theories that allow users to unblind the TI-signed tokens and recalculate values from the DLEQ proof (detailed in Figure 13 located in Appendix A), as well as letting the TI verify the user's challenge solution and check if a token has previously been spent.

The rules that define the actions of the participants in the protocol follow the sequence of the workflow diagrams in Figure 1. To simulate a loop behaviour that allows the user to redeem the 3 tokens at different times, PP utilizes the Store function (Figure 2) to store each token in the same rule (`Usr_verify_token`) and consume them one by one in the next rule `Usr_send_request` and use them through the rest of the token redemption phase rules, as shown in Figure 7.

C. Modelling Correctness and Security Properties for TAMARIN

It is necessary to adapt the cryptographic definitions from Section II-B into a definition that aligns with the formal reasoning of TAMARIN, ensuring that the concepts can be accurately represented and verified in the tool (up to the limitations of the Dolev-Yao model). In this section, we define the following security properties to be verified in TAMARIN.

1) ***C: Correctness****:* Requires that the protocol has a consistent execution running as expected if no adversary is present and any token produced in the issuance phase can successfully be validated in the redemption phase. To model this property in TAMARIN, we need to prove that all the rules are executed in the expected order

```
rule Usr_verify_token:
  let ... in
  [...] −− [...]−>[
    Store($Usr, $Srv, t, unblinded),
    Store($Usr, $Srv, t1, unblinded1),
    Store($Usr, $Srv, t2, unblinded2)
  ]
rule Usr_send_request:
  let ... in
  [ Store($Usr, $Srv, t, unblinded),... ]
  −−[...]−>[...]
```

Fig. 7: Usr_verify_token rule storing the signed unblinded tokens (top) and Usr_send_request rule consuming one of the stored tokens (bottom)

and the user is able to receive the requested resource when using a token signed by the TI.

2) ***SP1: One-More-Token Security****:* Given a successfully verified token that allows the server to send the requested resource back to the user, we require that this token has previously been signed by the TI with its secret key. Intuitively, this reflects the game-based one-more-token security definition (Definition 2) which requires an adversary to produce $n+1$ valid tokens, after only receiving n from the TI. We can interpret the ability of the cryptographic adversary $\mathcal{A}$ to query $\mathcal{O}_{\mathsf{TI\text{-}Issue}}$ and $\mathcal{O}_{\mathsf{TI\text{-}Redeem}}$ in the game-based definition as the TAMARIN adversary `A` learning outputs of the TI token signing and TI stored token verification rules. Then, `A` can disprove the lemma (which equates to $\mathcal{A}$ winning the game) by providing a valid token not signed by the TI.

3) ***SP2: Unlinkability****:* Given two tokens, an adversarial TI cannot determine whether they belong to the same user or different users. This captures the intuition of Figure 5, whereby an adversary wins if it is able to distinguish the two tokens. To replicate $\mathcal{A}$'s capability to query $\mathcal{O}_{\mathsf{U\text{-}Issue}_1}$, $\mathcal{O}_{\mathsf{Chal}}$, $\mathcal{O}_{\mathsf{U\text{-}Redeem}}$ and $\mathcal{O}_{\mathsf{U\text{-}Issue}_2}$, we allow `A` to executes rules to issue and redeem tokens. Finally, TAMARIN's observational equivalence represents $\mathcal{O}_{\mathsf{Chal}}$ and captures the essence of Definition 3 by requiring the distinction of two systems.

4) ***SP3: Single Signing Key****:* For this property, we define a seed `t` which is used to generate a blinded token `bt` and produce a user-verified signed token `st`. This property requires that prior to the user verification, `t` has been signed by the TI with its secret key `TIsk`. This reflects the game-based Definition 4, where an adversary wins the game if it can produce a valid signed token using a key different from `TIsk`. We enable `A` to achieve similar behaviour as querying $\mathcal{O}_{\mathsf{U\text{-}Issue}_1}$,$\mathcal{O}_{\mathsf{U\text{-}Issue}_2}$, by receiving information from the user rules which request resources and unblind signed tokens.

D. Verification of Correctness and Security

We now establish the correctness of our models and analyse them against the formalised security notions from Section III-C. We verified our models (which can be found at [39]) using TAMARIN v1.10.0 on a server with 2 Intel Xeon E5-2667 v3 CPUs (16 cores, 32 threads) and 384GB of RAM. Our results are summarised in Table II found in Section IV.

Correctness*:* To ensure the functional correctness of the Privacy Pass TAMARIN models (*C*), a `correctness` lemma is proven. Aided with first-order logic, it uses the trace produced upon running the protocol to ensure that the rules are executed and appear in a specific sequence determined by timepoints. We optimise the redemption phase section of the lemma for PP by checking only for the first and last rule of the redemption phase. To get from the initial to the last action, the protocol would have to execute all the intermediate rules. The secure channels ensure that the adversary cannot learn or modify the messages. The correctness lemma is auto-verified when the protocols are run, and the produced traces are manually checked using the interactive mode of TAMARIN.

One-More-Token Security*:* We define a lemma (Figure 8) in PP, covering *SP1*, that ensures that a TI entity is set up with its `secr_key`, and then at a later point, a token signed with `secr_key` is attempted to be redeemed. The lemma includes a clause that requires if a token is redeemed, then its seed must have been signed by the TI using its `secr_key` at a time point which occurs after the setup of the entity and before the token redemption.

TAMARIN proves this lemma by contradiction as it attempts to find a trace where a token TI-signing action does not exist before the redemption of the token. Because such a trace does not exist, the security property is successfully verified. Moreover, this lemma also demonstrates that tokens received from one TI cannot be successfully spent in a redemption phase with a different TI because the signing keys would not be the same. Subsequently, this one-more-token security property would not hold in the presence of mismatched signing keys in the actions of the TI setup, TI token issuance, and TI token verification rules.

Unlinkability: To prove unlinkability, we rely on observational equivalence and use the simplified models $\mathsf{PP}_{2.1}$ and $\mathsf{PP}_{2.2}$ to concentrate on the properties we want to show, i.e. the unlinkability of the user tokens and the adversary's inability to inject and manipulate tokens. To prove the latter, we include two trace lemmas in $\mathsf{PP}_{2.1}$, demonstrating that the adversary `A` can successfully generate tokens and impersonate the user (*A2.1*) but will be caught at the user token verification stage of the protocol (*A2.2*). Similarly, we show that `A` can maliciously sign tokens (*A2.3*) but again will be caught by the verification on the user side (*A2.4*).

To achieve the aforementioned properties, we collapse various PP rules in $\mathsf{PP}_{2.1}$ and $\mathsf{PP}_{2.2}$, and ensure that only the relevant tokens are leaked to the attacker to align with $\mathcal{A}$'s capabilities in Section II-B. This is because the assumption in PP is that the channels are encrypted and no tokens are leaked. However, $\mathsf{PP}_{2.1}$ allows the attacker to inject messages into the now insecure channel between the user and server to represent $\mathcal{A}$'s capabilities to initiate the issuance by sending out blinded tokens to the TI. Furthermore, we also permit `A` to sign blinded tokens that will be returned to the user, thus impersonating the TI in signing tokens. Moreover, to model an adversarial TI, we add a key leaking rule (Figure 10), and denote this attack model as D^*. Additionally, both models focus on the issuance phase and the computation of the different tokens since the redemption algorithm of the TI is only concerned with token validity.

We first model the generation of two blinded tokens by Usr_1 and one blinded token by Usr_2 in $\mathsf{PP}_{2.1}$. Since blinded tokens are computed using fresh seeds `t` and blinding factors `r`, and are not linked to a specific user, they can be generated in the same rule by the same actor. The TI-signed blinded tokens and DLEQ proofs are then sent back to the server securely to forward to the user, which it then broadcasts on the insecure channel.

We then use $\mathsf{PP}_{2.1}$ to verify the adversarial capabilities (*A2.1-A2.4*). The lemma covering *A2.1* – `adversary_generates_tokens`, follows the correctness lemma up to the TI signing the tokens. However, it expects the user-generated tokens `tokens1` (with their corresponding seeds `s`) to be different from the tokens `tokens2` received by the TI, meaning that the adversary can successfully impersonate a user to the TI if no checks are performed after the signing of the tokens. Subsequently, the `adversary_tokens_not_verified` lemma extends `adversary_generates_tokens` and checks that the user verification of the signed tokens (the `UnblindedTok` fact) is also present in the trace. The lemma checks that for all traces this is only possible if `tokens1` and `tokens2` are the same since the seeds, for which the proof is verified, have to be the same as `s`. TAMARIN proves that any adversarial tokens *A2.1* will be caught in the user check (*A2.2*) by verifying both lemmas.

```
lemma one_more_token_security:
"All tokenIssuer server user t Y secr_key res #i #k #l.
  Setup_Ti(tokenIssuer, Y, secr_key) @ i &
  TiVerifySignedToken(tokenIssuer, t, secr_key) @ k &
  SrvSendNewResource(server, user, res, t) @ l &
  #i < #k & #k < #l
  ==> ( Ex #j. TiSignsToken(tokenIssuer, t, secr_key)
        @ j & #j < #k)"
```

Fig. 8: one_more_token_security lemma verifying *SP1*

The lemma `adversary_injects_msgs` allows the adversary to impersonate the TI (*A2.3*) by only expecting a trace in which the user sends out their blinded tokens and later receives signed tokens from the server over the insecure channel. Following that, `adversary_injects_msgs_not_verified` checks that for all traces, these tokens will only be verified by the user if the TI has previously signed them (*A2.4*). TAMARIN verifies both lemmas, showing that `A` cannot successfully impersonate the TI to the user and provide tokens that pass the DLEQ check.

The model $\mathsf{PP}_{2.1}$ demonstrates that despite the channel between the user and server being insecure, the attacker is unable to successfully inject or manipulate the tokens. Therefore, to avoid non-termination when using observational equivalence, we create an identical model $\mathsf{PP}_{2.2}$ but with a secure User-Server channel. As shown by $\mathsf{PP}_{2.1}$, this modification does not limit the adversary's capabilities. Like $\mathsf{PP}_{2.1}$, at the end of $\mathsf{PP}_{2.2}$, the users receive the signed tokens, verify and unblind them, and compute the corresponding MAC values s. However, we use `diff(<Usr1t2,Usr1S2>, <Usr2t, Usr2S>)` to indicate the creation of two instances of $\mathsf{PP}_{2.2}$, for one of which `Usr1`'s values will be used as the second argument in the `Out` fact, and the other instance – `Usr2`'s. When this model is run, TAMARIN verifies that an adversary cannot distinguish between the two instances.

In addition to the secure communication between the actors in $\mathsf{PP}_{2.2}$, we also send out the user-generated blinded tokens and the TI-signed blinded tokens over the insecure channel to allow `A` access. Therefore, by the end of $\mathsf{PP}_{2.2}$, an adversary has access to the seeds t and be able to compute $f_{\mathsf{sk}}(T)$, and the blinded signed tokens – $f_{\mathsf{sk}}(\widetilde{T})$. The automatic mode of TAMARIN verifies the observational equivalence in $\mathsf{PP}_{2.2}$, meaning that despite the knowledge of those values, an adversary cannot distinguish the tokens of different users. Additionally, `A` cannot link the blinded tokens (used in the issuance phase) to the seeds (used in the redemption phase), which is consistent with the game-based Definition 3 of the property. From these results, it can be concluded that the issuance and redemption phases cannot be linked since the blinded token does not reveal information about `t` and cannot be associated with the signed token either, thus we achieve unlinkability.

Single Signing Key: The trace lemma (Figure 9) for the *SP3* property ensures that there is a TI instance created with its `secr_key`. A user initiates the token issuance phase by sending a blinded token `bt` computed from the seed `t`, denoted by the `UsrCreateToken` action fact. Further, the lemma requires that if a signed token from the same `t` is verified and accepted by the user, then the TI must have signed `bt` (with `secr_key`) after the User

has initiated the issuance and before the signed token is received back. TAMARIN successfully verifies the lemma with the help of an oracle, thus formally proving that the user only accepts tokens signed with the active `secr_key`.

IV. Stronger Attack Models

Inspired by the discussion on the concerns of TI key leakage during deployment by designers [6, 19] and the IETF [18], we consider a threat model that permits the adversary the capability to corrupt a TI signing key, and subsequently enables it to eavesdrop and inject messages into the secure channels between actors. This is motivated by the high impact of such a leakage, disrupting potentially hundreds of thousands of users. As mentioned previously, existing tools to limit impact such as key rotation and revocation are slow and disruptive. Furthermore, recovery may be a cause of additional friction for users (who can no longer spend their honestly-obtained tokens) and may even enable indirect tracking attacks through the increase fingerprinting surface and behavioural correlation observed from additional CAPTCHA interactions.

We implemented this in TAMARIN by allowing `A` to gain knowledge of the channel identifier and the message being sent. Then, using the identifier, it can construct a new message to back into the secure channel. To model this in TAMARIN, we add a rule to model a malicious TI that can leak its `secr_key` (Figure 10). We do, however, prohibit this behaviour in the correctness lemma, by adding a condition that the found trace does not include injections into the channel. These modifications allow us to verify the security properties of Privacy Pass when the attacker has knowledge of the TI's `secr_key`. While we were able to autoprove most lemmas, some needed to be manually verified. Our results are given in Table II.

A. Attack on One-More Token Security

When running the TAMARIN models with the stronger threat model, a potential attack on the one-more-token security property (*SP1*)

```
lemma single_signing_key:
"All tokenIssuer server user Y secr_key t u comm sol bt
  #t01 #t02 #t03.
  Setup_Ti(tokenIssuer, Y, secr_key) @ t01 &
  UsrCreateToken(user, server, tokenIssuer, sol, bt, t)
    @ t02 &
  UsrVerifyAndStoreToken(user, server, t, u, comm) @ t03 &
  #t01 < #t02 & #t02 < #t03
  ==> ( Ex #i . TiSignsToken(ticketIssuer, t, secr_key)
        @ i & #t02 < #i & #i < #t03 )
```

Fig. 9: single_signing_key lemma verifying *SP3*

```
rule Ti_leaked_signing_key:
  [ LeakKeys(Ti, secr_key) ]
  --[ TiLeakedKey(Ti, secr_key) ]->
  [ Out(secr_key) ]
```

Fig. 10: Ti_leaked_signing_key rule to leak the secret key of the TI

was discovered. Definition 2 requires that every token that is being redeemed must have previously been signed by the TI with its `secr_key`. However, due to the presence of a malicious TI that leaks its signing key, the adversary is able to produce a token T and sign it to produce $f_{\mathsf{sk}}(T)$. `A` then captures the channel identifier for the communication channel, after the server forwards the valid TI-signed tokens to the user. Having that identifier, the attacker also produces a URL request and, impersonating a user, sends a request for that URL to the server. Once knowledge of the challenge is acquired by `A` from the secure channel, it can construct a message to be sent back to the server that would include the challenge, requested URL, random `t` for which the adversary had signed $f_{\mathsf{sk}}(T)$, and the MAC s computed for the token.

The verification on the TI's side passes as the adversary's seed had not been used before and the recomputed MAC is equal to the one produced by `A`. Hence, the TI accepts the token without having signed it previously, which is a valid attack against the protocol's one-more-token security property.

Whilst this attack is unsurprising, identification of such an attack does validate that our model is faithful to the protocol and builds confidence that our security analysis in security model D presented in Section III-D is correct. Furthermore it reinforces previous discussion that key leakage is indeed a valid concern when deciding key rotation frequencies, with potentially a large impact for web services.

V. Privacy Pass Plus

In this section, we propose a new protocol—Privacy Pass Plus—that offers stronger security against key leakage from the token issuer. In our new protocol, we are able to mitigate this threat by reducing reliance on key rotation, and allowing the server an active role in the protocol to 'spot check' tokens. This is done by cryptographically allowing the server to determine whether it was involved in issuing a token being spent, thus even in the event of a TI key leak, an adversary cannot successfully generate valid tokens. This mechanism enhances security by introducing an additional layer of verification, making token issuance and validation more robust.

While our protocol introduces computational work for the server, this is offset by its flexible implementation: the policy that defines whether the check is performed is entirely decided and implemented by the server. For example, it can employ the check selectively, regularly as a spot-check mechanism, or in response to anomalies such as an abnormal influx of requests. Indeed, the server checking every token request would render the TI and Privacy Pass

TABLE II: Results for the security properties of the TAMARIN models for Privacy Pass.

Model	Run Time (s)	SP	Verified	Heuristic	AM
PP	Total (*C*, *SP3*): 161 (S)	*C*	✓	built-in	S
	Total (*C*, *SP1*, *SP3*): 128 (D)	*SP1*	✓	built-in	D
		SP1	✗	manual	S
		SP3	✓	oracle	S
$\mathsf{PP}_{2.1}$	Total (*C*, *A3.1 - A3.4*): 122	*C*	✓	built-in	D*
		A3.1-A3.4	✓	oracle	D*
$\mathsf{PP}_{2.2}$	Total (*C*, *SP2*): 3999	*C*	✓	built-in	D*
		SP2	✓	oracle	D*

Legend: *SP* = Security Property, *AM* = Attack Model, *D* = Default attack model, *D** = Default attack model with TI key leakage, *S* = Stronger attack model.

protocol redundant, so this additional verification is only expected to check some orders of magnitude less than that of the TI. We also ensure the server check can only happen after the TI evaluates and confirms a successful redemption, which allows the TI to handle flooding and DoS attacks as intended (for when the signing key is not leaked). Finally, we note that if a server does not ever perform its check, we simply recover the original Privacy Pass functionality.

The design for our scheme is influenced by the standard construction of Privacy Pass, and utilises much of the same cryptographic mechanisms. This allows for easier adoption in the Privacy Pass ecosystem. By addressing critical trade-offs in Privacy Pass, our Plus variation not only improves resistance to key leakage but also empowers servers with a more active role in securing the protocol. This shift in responsibility distributes trust more evenly between the TI and server, reducing the system's reliance on a single point of failure and providing a pathway to stronger guarantees for robust security.

However, it should be noted that one benefit of Privacy Pass is that tokens obtained in a request can be spent in another at a different server, provided the TI is the same. This is now not possible when including the server in the redemption process. We argue, however, that in practice this has limited impact as it is often the case that the TI, whilst distinct from a modelling point of view, is practically implemented by the same service. For example, Cloudflare operating both the CDN Server and its own TI. Furthermore, it is noted in the IETF that distributed users could obtain cacheable tokens and them share them with a single user to redeem in a way that would violate a server's attempt to limit tokens to any one particular user, in what's known as a hoarding attack. Privacy Pass Plus would limit this attack and prevent users from caching tokens across multiple servers.

A. Privacy Pass Plus Construction

Intuitively, the protocol for Privacy Pass Plus (PP^+) is inspired by PP, with an additional step performed by the Server – we state the formal description in Figures 11 and 12. We now give an overview, but due to similarities with PP we focus on the changes, and assume notation and algorithms from Section II. In particular, our PP^+ follows the procedural flow from Figure 1.

Protocol Setup. Define an algorithm Setup that takes as input a security parameter λ and returns the public parameters pp, which consists of the description of the group $\mathbb{G}$. The key generation algorithm KeyGen that takes as input the public parameters pp and returns a secret key $\mathsf{sk} \leftarrow\!\$\, \mathbb{Z}_q$. Using a commitment scheme, it outputs a commitment c_{sk} to sk. The Server also computes a secret key $\mathsf{rk} \leftarrow\!\$\, \mathbb{Z}_q$.

Token Issuance. The protocol executes identically to that presented in Figure 2 except that the server, upon receiving a signed blinded token $f_{\mathsf{sk}}(\widetilde{T})$ from the TI, also performs a sign operation over this value to create $f_{\mathsf{rk}}(f_{\mathsf{sk}}(\widetilde{T}))$, which can be thought of as a token signed by both TI and the Server. This is appended to the TI token and also sent to the User. To complete the issuance, the user unblinds both $f_{\mathsf{sk}}(\widetilde{T})$ and $f_{\mathsf{rk}}(f_{\mathsf{sk}}(\widetilde{T}))$ to recover tokens $f_{\mathsf{sk}}(T)$ and $f_{\mathsf{rk}}(f_{\mathsf{sk}}(T))$, which they store in the stack.

Token Redemption. Again, the protocol executes as described in Figure 3, except initially the User also computes a MAC tag s_2 for the 'double-signed' token $f_{\mathsf{rk}}(f_{\mathsf{sk}}(T))$. It sends the seed t, s_1, and s_2 to the Server, which forwards it on. Having checked and accepted the token, the TI now returns the signed token $f_{\mathsf{sk}}(T)$ to the server. This is an additional communication in our protocol. If the check-bit cb is not set (decided by some server-specified policy), then the server simply serves the request to the user. Else, it checks that the token is valid under its key, by computing $f_{\mathsf{rk}}(f_{\mathsf{sk}}(T))$ from $f_{\mathsf{sk}}(T)$, and re-deriving the MAC key as $k_2 \leftarrow \mathsf{sKGen}(t, f_{\mathsf{rk}}(f_{\mathsf{sk}}(T)))$. This allows the server to check validity with respect to its key against s_2. It outputs 1 if this is satisfied and the response from the TI was also valid, else it sends 0.

B. Cryptographic Security Analysis of Privacy Pass Plus

We now show that our construction for Privacy Pass Plus meets the cryptographic computational properties of the stronger computational attack model. These definitions are formally presented in Appendix E. Intuitively, the main difference is in the one-more-token security, which now allows the adversary playing the role of the user to corrupt the TI, i.e. obtain knowledge of its secret key. All other properties, including correctness, capture the same capabilities of the adversary and only differ in that the algorithm modelling has minor differences. For conciseness, and without loss in generality, we assume the role of the server is incorporated into the $\mathsf{TI\text{-}Issue}$ and $\mathsf{TI\text{-}Redeem}$ phases, i.e., the TI and S are honest in the one-more-token, and malicious in the unlinkability experiments. The exception is in the single signing key experiment, where only the TI is malicious, and so the server's role is included in $\mathsf{U\text{-}Issue}_1$, $\mathsf{U\text{-}Issue}_2$ and $\mathsf{U\text{-}Redeem}$. Note that proofs for Theorems 2 to 4 are given in Appendix E1.

Theorem 1. PP^+ *is correct (according to Definition 14) if* NIZK *and* MAC *are correct.*

Proof. Without loss of generality, assume that $\mathsf{cb} = 1$. Let $(T, r) \leftarrow\!\$\, \mathsf{Blind}(\mathsf{GenToken}(t))$ where $t \leftarrow\!\$\, \mathbb{Z}_q$, therefore $\widetilde{T} = T^r$. When TI receives $\widetilde{T}$ it first computes $\widetilde{T}^{\mathsf{sk}} = f_{\mathsf{sk}}(\widetilde{T}) \leftarrow \mathsf{Sign}(\widetilde{T}, \mathsf{sk})$. It then computes a DLEQ NIZK proof $\pi_{\mathsf{sk}} \leftarrow \mathsf{Prove}(\mathsf{sk}, c_{\mathsf{sk}}, f_{\mathsf{sk}}(\widetilde{T}))$ and sends $(\widetilde{T}^{\mathsf{sk}}, \pi_{\mathsf{sk}})$ back to U. The server S then computes $\mathsf{Sign}(\mathsf{rk}, f_{\mathsf{sk}}(\widetilde{T})) = \widetilde{T}^{\mathsf{sk}\cdot\mathsf{rk}}$. The tuple $(\widetilde{T}^{\mathsf{sk}}, \widetilde{T}^{\mathsf{sk}\cdot\mathsf{rk}}, \pi_{\mathsf{sk}})$ is sent to the user. Since π_{sk} is a valid DLEQ NIZK proof, then $1 \leftarrow \mathsf{Verify}(\pi_{\mathsf{sk}}, c_{\mathsf{sk}}, \widetilde{T}, \widetilde{T}^{\mathsf{sk}})$. Therefore, the user computes $T^{\mathsf{sk}} \leftarrow \mathsf{Unblind}(\widetilde{T}^{\mathsf{sk}}, r)$ and the signing phase is complete. For the redemption phase, U computes two shared keys $K_1 \leftarrow H_2(t, T^{\mathsf{sk}})$ and $K_2 \leftarrow H_2(t, T^{\mathsf{sk}\cdot\mathsf{rk}})$, it sends $(t, \mathsf{MAC}_{K_1}(R_{TI}), \mathsf{MAC}_{K_2}(R_S))$ to S, who forwards $(t, \mathsf{MAC}_{K_1}(R_{TI}))$ to TI. Given t, TI calculates $T^{\mathsf{sk}} = \mathsf{Sign}(\mathsf{GenToken}(t))$ and computes $\mathsf{MAC}_{H_2(t, T^{\mathsf{sk}})}(R_S)$. Since $K_1 = H_2(t, T^{\mathsf{sk}})$ by definition, we conclude that $\mathsf{MAC}_{K_1}(R_{TI}) = \mathsf{MAC}_{H_2(t, T^{\mathsf{sk}})}(R_{TI})$. Then TI forwards its response and the token T^{sk} to S, and assuming without loss in generality that $\mathsf{cb} = 1$, then the Server computes $T^{\mathsf{sk}\cdot\mathsf{rk}}$ and $\mathsf{MAC}_{H_2(t, T^{\mathsf{sk}\cdot\mathsf{rk}})}(R_S)$. Since $K_2 = H_2(t, T^{\mathsf{sk}\cdot\mathsf{rk}})$ then we also conclude that $\mathsf{MAC}_{K_2}(R_S) = \mathsf{MAC}_{H_2(t, T^{\mathsf{sk}\cdot\mathsf{rk}})}(R)$, and the response sent to the User is 1, which establishes correctness. □

Theorem 2. PP^+ *enjoys unlinkable+ security (Definition 15).*

Theorem 3. PP^+ *has one-more-token+ security (Definition 16). in the Random Oracle Model if El Gamal is secure against one-more decryption attacks and the* NIZK *proof system is zero-knowledge.*

Theorem 4. PP^+ *has the single-signing-key+ property (Definition 17). if the zero-knowledge proof system* NIZK *is sound and the commitment scheme* Com *has the binding property.*

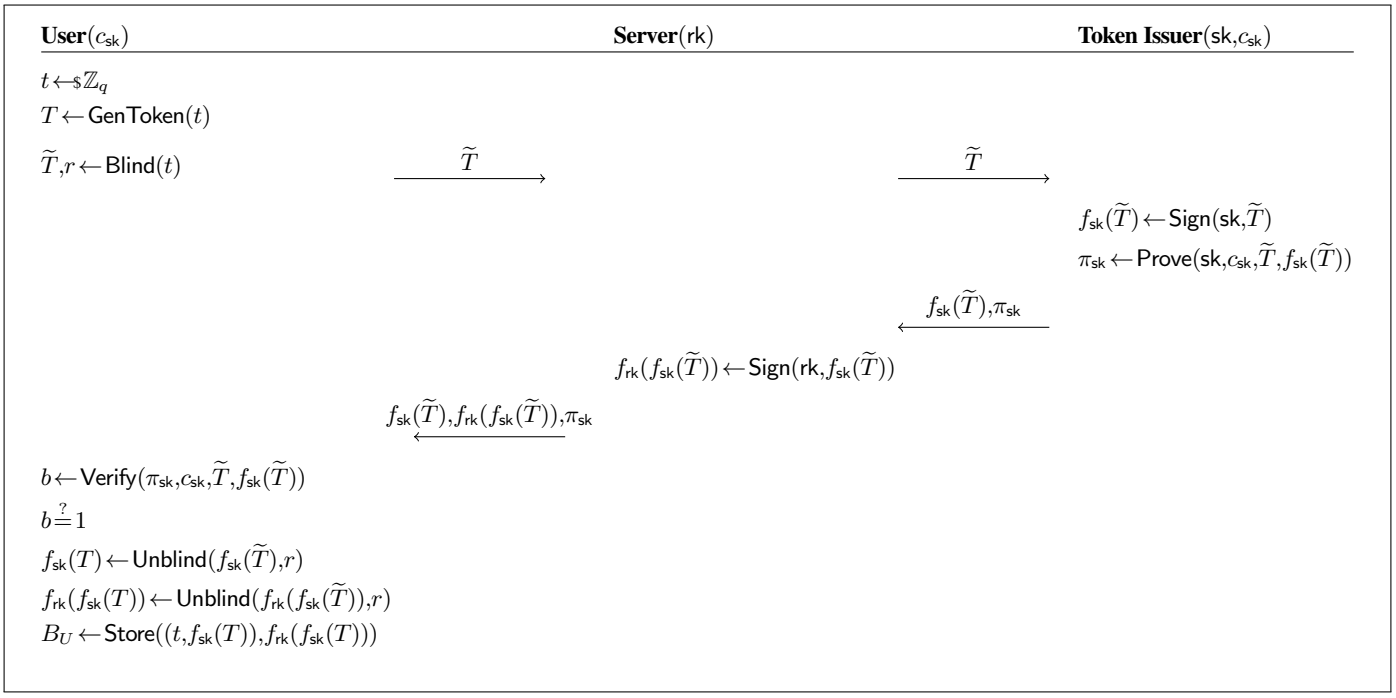


Fig. 11: Privacy Pass Plus token issuance

C. Formal Verification of PP^+

We now show that Privacy Pass Plus is secure against the stronger threat model (Section IV), denoted as S, in which the adversary can eavesdrop, inject messages, and has knowledge of the TI secret key. We build the TAMARIN model of PP^+ [39] by extending the PP model. To accurately represent the new protocol, we introduce a server secret signing key (`srv_secr_key`), used by the server in the token issuance phase (Figure 11) to sign the TI-signed token $f_{sk}(\widetilde{T})$, resulting in $f_{rk}(f_{sk}(\widetilde{T}))$. Additionally, our PP^+ model specifically addresses scenarios in which the Server detects anomalies and consequently performs spot-checking. In cases where no abnormalities are detected, the redemption phase flow is consistent with the original Privacy Pass protocol.

The changes in the token redemption phase of PP^+ (Figure 12) include the computation of a shared key k_2 and MAC s_2 for the token signed both by the TI and the Server - $f_{rk}(f_{sk}(T))$, in addition to k_1 and s_1, generated for $f_{sk}(T)$. This calculation includes the newly introduced value R_S, which, similarly to the R in PP (referred to as R_{TI} in PP^+), both the server and user share knowledge of. Furthermore, if the recomputed by the TI MAC s_1' for $f_{sk}(T)$ is the same as the user's s_1, the TI sends the signed unblinded token $f_{sk}(T')$ back to the server in addition to res_1. Then the Server recomputes the shared key k_2' and MAC s_2' for $f_{rk}(f_{sk}(T))$ and compares it with s_2 sent by the User in their response to the server challenge. If the MAC values are the same, the Server sends the requested resource back to the user.

We also ensure the models $\mathsf{PP}_{2.1}$ and $\mathsf{PP}_{2.2}$, proving *A2.1-A2.4* and *SP2*, adhere to the changes in the PP^+ TAMARIN model. Thus, we built new models $\mathsf{PP}^+_{2.1}$ and $\mathsf{PP}^+_{2.2}$, which include a server secret signing key and the token signing done by the server after receiving the token signed by the TI ($f_{sk}(T)$) in our model simplifications.

Following the aforementioned changes, we run the same lemmas we do for PP, $\mathsf{PP}_{2.1}$, and $\mathsf{PP}_{2.2}$ on the modified model PP^+ (under attack model S) and the simplifications $\mathsf{PP}^+_{2.1}$ and $\mathsf{PP}^+_{2.2}$ (under attack model D^*). All lemmas and observational equivalence were successfully proven using either a proof oracle or the standard TAMARIN heuristics. The results for the Privacy Pass Plus models are shown in Table III. They demonstrate that the changes implemented to produce the PP^+ protocol do not affect the correctness, single signing key, and unlinkability lemmas, and they are all verified by TAMARIN.

Additionally, the one-more-token security property is also successfully proven for the PP^+ model under the S attack model, described in Section IV. The discovered attack on *SP1* of PP is no longer viable as the attacker is not able to produce a valid token signed with both the TI's and server's secret signing keys that would pass the secondary check by the server. This result shows that our protocol Privacy Pass Plus is one-more-token secure in S,where the adversary can inject messages into the secure channel and can use the TI's leaked secret signing key. As demonstrated in Table III, all of the security properties of PP^+ are successfully verified in the stronger attack model S, which also implies the properties also hold in D.

TABLE III: Results of the TAMARIN models for Privacy Pass Plus.

Model	Run Time (s)	SP	Verified	Heuristic	AM
PP^+	Total (*C*, *SP1*, *SP3*): 1717	*C*, *SP1*, *SP3*	✓	oracle	S
$\mathsf{PP}^+_{2.1}$	Total (*C*, *A3.1 - A3.4*): 618	*C*	✓	oracle	D*
		A3.1-A3.4	✓	built-in	D*
$\mathsf{PP}^+_{2.2}$	Total (*C* and *SP2*): 3771	*C*, *SP2*	✓	oracle	D*

Legend: *SP* = Security Property, *AM* = Attack Model,
*D** = Default attack model with TI key leakage, *S* = Stronger attack model.

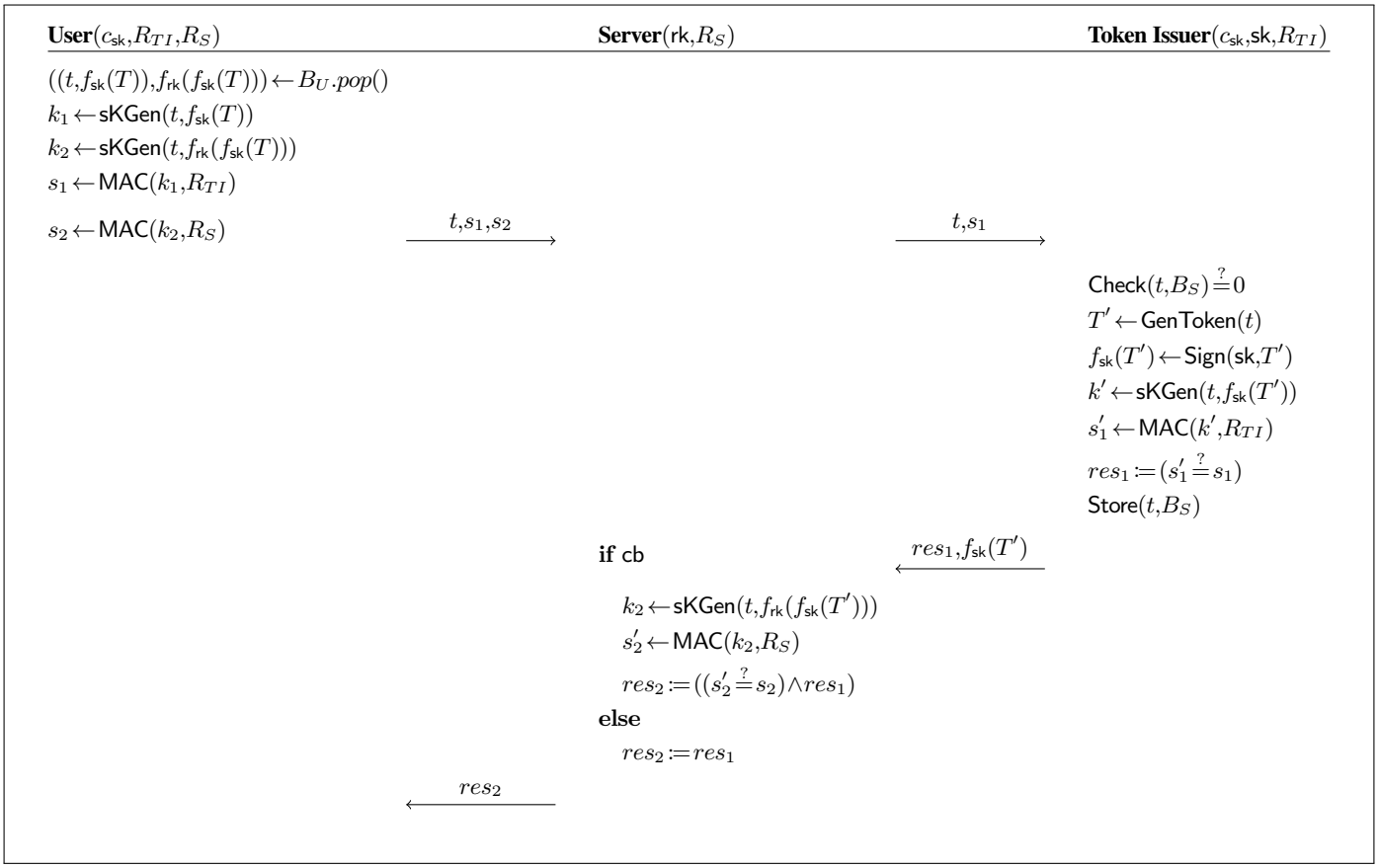


Fig. 12: Privacy Pass Plus token redemption.

VI. Conclusions

In this paper, we present the first formalisation of the Privacy Pass protocol in the symbolic modelling tool Tamarin and use it to verify the protocol's security guarantees, interpreted from our game-based definitions. We successfully verify the trace lemmas for one-more-token security and single signing key, and use observational equivalence to prove token unlinkability. When analysing the protocol in Tamarin, we identify the one-more-token security property is violated under a stronger threat model allowing TI key leakage. Whilst this attack model is strong, it has been highlighted in related works and the IETF as a potential high-impact exploitation and thus benefits from analysis. To mitigate the identified attack, we proposed a new protocol—Privacy Pass Plus—which prevents the attack by introducing a spot-checking mechanism that the Server executes. The new protocol was also modelled in Tamarin, and the security guarantees of the original protocol still hold under the stronger attack model, and the discovered attack was no longer viable. Finally, we provide cryptographic reductions for Privacy Pass Plus to prove it satisfies the security guarantees defined by the creators of Privacy Pass. Together, cryptographic proofs and formal verification in Tamarin complement each other, ensuring a comprehensive analysis of the protocol's security.

References

[1] imperva.com, "2025 Bad Bot Report | Resource Library — imperva.com," https://www.imperva.com/resources/resource-library/reports/2025-bad-bot-report/, n.d., [Accessed 16.10.2025].

[2] DataDome, "Global Bot Security Report 2024," https://datadome.drift.click/a430172e-2941-4683-bf0f-f0c3dfc2800e, 2024, [Online; Accessed 10.08.2025].

[3] L. von Ahn, M. Blum, and J. Langford, "Telling humans and computers apart automatically," *Commun. ACM*, 2004.

[4] A. Reddy and Y. Cheng, "User Perceptions of CAPTCHAs: University vs. Internet Users," in *DBSec 2024*, 2024.

[5] support.torproject.org, "Google makes me solve a Captcha or tells me I have spyware installed | Tor Project | Support — support.torproject.org," https://support.torproject.org/tbb/tbb-44/, n.d., [Online; Accessed 02.08.2024].

[6] A. Davidson, I. Goldberg, N. Sullivan, G. Tankersley, and F. Valsorda, "Privacy pass: Bypassing internet challenges anonymously," *PETS 2018*, 2018.

[7] S. Celi, S. Davidson, Alex Valdez, and C. A. Wood, "RFC 9578: Privacy Pass Issuance Protocols — rfc-editor.org," https://www.rfc-editor.org/rfc/rfc9578.html, 2024.

[8] chromewebstore.google.com, "Silk - Privacy Pass Client - Chrome Web Store," https://chromewebstore.google.com/detail/silk-privacy-pass-client/ajhmfdgkijocedmfjonnpjfojldioehi?hl=en, n.d., [Accessed 12.08.2025].

[9] B. Blanchet, "Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif," *Found. Trends Priv. Secur.*, 2016.

[10] D. Baelde, S. Delaune, C. Jacomme, A. Koutsos, and J. Lallemand, "The Squirrel Prover and its Logic," *ACM SIGLOG News*, 2024.

[11] A. Armando, D. Basin, Y. Boichut, Y. Chevalier, L. Compagna, J. Cuellar, P. H. Drielsma, P. C. Heám, O. Kouchnarenko, J. Mantovani, S. Mödersheim, D. von Oheimb, M. Rusinowitch, J. Santiago, M. Turuani, L. Viganò, and L. Vigneron, "The AVISPA Tool for the Automated Validation of Internet Security Protocols and Applications," in *CAV 2005*, 2005.

[12] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN Prover for the Symbolic Analysis of Security Protocols," in *CAV*, 2013.

[13] S. Celi, J. Hoyland, D. Stebila, and T. Wiggers, "A Tale of Two Models: Formal Verification of KEMTLS via Tamarin," in *ESORICS*, 2022.

[14] C. Cremers, M. Horvat, J. Hoyland, S. Scott, and T. van der Merwe, "A Comprehensive Symbolic Analysis of TLS 1.3," in *ACM SIGSAC CCS*, 2017.

[15] H. Feng, H. Li, X. Pan, and Z. Zhao, "A formal analysis of the FIDO UAF protocol," in *NDSS 2021*, 2021.

[16] D. Basin, J. Dreier, L. Hirschi, S. Radomirovic, R. Sasse, and V. Stettler, "A Formal Analysis of 5G Authentication," in *ACM SIGSAC CCS*, 2018.

[17] R. Miller, I. Boureanu, S. Wesemeyer, and C. J. P. Newton, "The 5G Key-Establishment Stack: In-Depth Formal Verification and Experimentation," in *ACM ASIA CCS*, 2022.

[18] A. Davidson, J. Iyengar, and C. A. Wood, "Privacy Pass Architectural Framework," https://datatracker.ietf.org/doc/draft-ietf-privacypass-architecture/03/, Tech. Rep., 2022.

[19] N. Tyagi, S. Celi, T. Ristenpart, N. Sullivan, S. Tessaro, and C. A. Wood, "A Fast and Simple Partially Oblivious PRF, with Applications," in *EUROCRYPT*, 2022.

[20] M. Abadi and P. Rogaway, "Reconciling two views of cryptography (the computational soundness of formal encryption)," *J. Cryptol.*, vol. 20, no. 3, p. 395, Jul. 2007.

[21] B. Blanchet, "Symbolic and computational mechanized verification of the arinc823 avionic protocols," in *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*, 2017, pp. 68–82.

[22] H. Comon-Lundh, V. Cortier, and G. Scerri, "Security proof with dishonest keys," in *Principles of Security and Trust*, P. Degano and J. D. Guttman, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 149–168.

[23] S. Hendrickson, J. Iyengar, T. Pauly, S. Valdez, and C. A. Wood, "Rate-Limited Token Issuance Protocol," https://datatracker.ietf.org/doc/draft-ietf-privacypass-rate-limit-tokens/06/, 2024.

[24] H. Chu, K. Do, and L. Hanzlik, "On the Security of Rate-limited Privacy Pass," in *ACM SIGSAC CCS*, 2023.

[25] K. Hanff, A. Lehmann, and C. Özbay, "Security analysis of privately verifiable privacy pass," ACM SIGSAG CCS, 2025.

[26] J. B. Almeida, M. Barbosa, G. Barthe, M. Campagna, E. Cohen, B. Grégoire, V. Pereira, B. Portela, P. Strub, and S. Tasiran, "A machine-checked proof of security for AWS Key Management Service," in *ACM SIGSAC CCS*, 2019.

[27] V. Cortier, D. Galindo, and M. Turuani, "A Formal Analysis of the Neuchatel e-Voting Protocol," in *IEEE EuroS&P*, 2018.

[28] D. Basin, S. Radomirović, and L. Schmid, "Dispute Resolution in Voting," in *IEEE CSF*, 2020.

[29] B. Lipp, B. Blanchet, and K. Bhargavan, "A Mechanised Cryptographic Proof of the WireGuard Virtual Private Network Protocol," in *IEEE EuroS&P*, 2019.

[30] N. Kobeissi, K. Bhargavan, and B. Blanchet, "Automated Verification for Secure Messaging Protocols and Their Implementations: A Symbolic and Computational Approach," in *IEEE EuroS&P*, 2017.

[31] C. Cremers, M. Horvat, S. Scott, and T. van der Merwe, "Automated Analysis and Verification of TLS 1.3: 0-RTT, Resumption and Delayed Authentication," in *IEEE S&P*, 2016.

[32] X. Hofmeier, "Formal Analysis of Web Single-Sign On Protocols using Tamarin," Bachelor's Thesis, 2019.

[33] L. Schrempp, R. Sasse, D. Basin, and S. Radomirović, "Formal Verification of FIDO2 with Human Interaction," Master's thesis, 2023.

[34] M. R. Albrecht and K. D. Gur, "Verifiable oblivious pseudorandom functions from lattices: Practical-ish and thresholdisable," in *ASIACRYPT 2024*.

[35] M. R. Albrecht, A. Davidson, A. Deo, and D. Gardham, "Crypto dark matter on the torus - oblivious prfs from shallow prfs and TFHE," in *EUROCRYPT*, 2024.

[36] D. Dolev and A. Yao, "On the security of public key protocols," *IEEE Trans. Inf. Theory*, 1983.

[37] T. Meunier, C. D. Rubin, and A. Faz-Hernández, "Privacy Pass: upgrading to the latest protocol version

— blog.cloudflare.com," https://blog.cloudflare.com/privacy-pass-standard, 2024, [Online; Accessed 03.08.2024].
[38] D. Jackson, *Software Abstractions - Logic, Language, and Analysis*. MIT Press, 2006.
[39] Anonymous Authors, "Tamarin model for privacy pass," https://anonymous.4open.science/r/privacyPass-F58E/README.md, 2025.
[40] D. Chaum and T. P. Pedersen, "Wallet Databases with Observers," in *CRYPTO*, 1992.
[41] A. Fiat and A. Shamir, "How To Prove Yourself: Practical Solutions to Identification and Signature Problems," in *CRYPTO*, 1986.
[42] R. Henry and I. Goldberg, "Batch Proofs of Partial Knowledge," in *ACNS*, 2013.

APPENDIX

In Sections A.1-A.4, we present the key concepts and building blocks of the Privacy Pass protocol.

A. Zero-Knowledge Proofs

For a given relation R, a language is defined as $L_R := \{x : \exists w\ (x,w) \in R\}$. A zero-knowledge proof system allows a prover to, for a statement x, prove knowledge of a witness w such that (x,w) belong to the relation R, without leaking any information about w.

Definition 5 (Non-Interactive Zero-Knowledge Proof System)**.** *A* NIZK *proof system for a relation R is a tuple of algorithms* (Setup,Prove,Verify,SimSetup,Sim) *defined as follows.*

- $\mathsf{Setup}(\lambda)$ *takes as input the security parameter λ and outputs a common reference string ρ.*
- $\mathsf{Prove}(\rho,x,w)$ *takes a common reference string ρ, statement x and witness w, returning a proof π.*
- $\mathsf{Verify}(\rho,x,\pi)$ *outputs 1 if π is a valid proof with respect to the common reference string ρ and statement x, and 0 otherwise.*
- $\mathsf{SimSetup}(\lambda)$ *takes as input the security parameter λ and outputs a common reference string ρ and a trapdoor τ.*
- $\mathsf{Sim}(\rho',x)$ *takes a common reference string ρ', statement x and returns a simulated proof π'.*

A NIZK proof system is *correct* if for any $\rho \leftarrow\!\$\ \mathsf{Setup}(\lambda)$ and $(x,w) \in R$ it follows that with overwhelming probability we have $1 \leftarrow \mathsf{Verify}(\rho,x,\mathsf{Prove}(\rho,x,w))$.

Definition 6 (Soundness)**.** *We say that a* NIZK *proof system is sound if the following advantage is negligible in the security parameter λ.*

$$\mathsf{Adv}^{\text{sound}}_{\mathcal{A},\mathsf{NIZK}}(\lambda) := \Pr\left[\begin{array}{c|c} \rho \leftarrow \mathsf{NIZK.Setup}(\lambda); & \mathsf{NIZK.Verify}(\rho,x,\pi) = 1 \\ (x,\pi) \leftarrow\!\$\ \mathcal{A}(\rho) & \wedge \quad x \notin L_R \end{array}\right]$$

Definition 7 (Zero Knowledge)**.** *A* NIZK *proof system has the zero-knowledge property if the following advantage is negligible in the security parameter λ.*

$$\mathsf{Adv}^{\text{zk}}_{\mathcal{A},\mathsf{NIZK}}(\lambda) := \left|\begin{array}{l} \Pr\left[\rho \leftarrow \mathsf{NIZK.Setup}(\lambda) : \mathcal{A}^{\mathsf{NIZK.Prove}(\rho,\cdot,\cdot)}(\rho) = 1\right] \\ -\Pr\left[(\rho',\tau) \leftarrow \mathsf{SimSetup}(\lambda) : \mathcal{A}^{\mathsf{Sim}(\rho',\tau,\cdot)}(\rho') = 1\right] \end{array}\right|$$

NIZK for Privacy Pass. Privacy Pass uses a non-interactive zero-knowledge proof defined by Chaum and Pedersen [40] in conjunction with the Fiat-Shamir transformation [41], which we recall in Figure 13.

Let $\mathbb{G}$ be a group with prime order q and generators X and P, then let $Y = X^{sk}$ and $Q = P^{sk}$ for some secret key $sk \leftarrow\!\$\ \mathbb{Z}_q$ only known to the prover. Define a hash function H_3: $\mathbb{G}^6 \rightarrow \mathbb{Z}_q$. The protocol in Figure 13 describes a NIZK proof of *Discrete-log Equality* (DLEQ), i.e., a proof for the relation $log_X Y = log_P Q$. We note that, as mentioned in Davidson et al. [6], this proof can be batched using techniques by Henry and Goldberg [42].

Server: X,Y,P,Q,sk,q		**Client:** X,Y,P,Q,q
$t \leftarrow\!\$\ \mathbb{Z}$		
$A = X^t, B = P^t$		
$c = H_3(X,Y,P,Q,A,B)$		
$s = (t - csk) \mod q$		
	$\xrightarrow{(c,s)}$	
		$A' = X^s Y^c, B' = P^s Q^c$
		$c' = H_3(X,Y,P,Q,A',B')$
		$c \stackrel{?}{=} c'$

Fig. 13: DLEQ non-interactive zero-knowledge proof

B. Public Key Encryption

Definition 8 (Public Key Encryption)**.** *A public key encryption scheme is a tuple of algorithms* (KeyGen, Enc, Dec)*, defined as follows.*

- $\mathsf{Setup}(\lambda)$ *takes as input the security parameter λ and outputs public parameters pp, which are implicit inputs to all algorithms.*
- $\mathsf{KeyGen}(\lambda)$ *takes as input the security parameter λ and outputs a key pair* (sk,pk).
- $\mathsf{Enc}(\mathsf{pk},m)$ *takes a public key* pk *and message $m \in \mathbb{M}$ and outputs the ciphertext c.*
- $\mathsf{Dec}(\mathsf{sk},c)$ *outputs m if c is a valid ciphertext with respect to the secret key* sk.

A PKE scheme is *correct* if for some $pp \leftarrow \mathsf{Setup}(\lambda)$ and any $(\mathsf{sk},\mathsf{pk}) \leftarrow\!\$\ \mathsf{KeyGen}(\lambda)$, it follows that with overwhelming probability we have $m \leftarrow \mathsf{Dec}(\mathsf{sk},\mathsf{Enc}(\mathsf{pk},m))$.

Definition 9 (One-More-Decryption)**.** *We say that a* PKE *has the One-More-Decryption property if the experiment defined in Figure 14 is negligible in the security parameter λ.*

$$\mathsf{Adv}^{\text{one-more-dec}}_{\mathcal{A},\mathsf{PKE}}(\lambda) := \mathsf{Exp}^{\text{one-more-dec}}_{\mathcal{A},\mathsf{PKE}}(\lambda) \leqslant \varepsilon(\lambda)$$

PKE for Privacy Pass. The security of Privacy Pass is reduced to the one-more-decryption property of the El Gamal PKE. As our construction will also use this instantiation, we recall the scheme in Definition 10.

Definition 10 (El Gamal Public Key Encryption)**.** *A public key encryption scheme is a tuple of algorithms* (KeyGen, Enc, Dec)*, defined as follows:*

$\mathsf{Exp}^{\text{one-more-dec}}_{\mathcal{A},\mathsf{PKE}}(\lambda)$

1: $pp \leftarrow \Pi.\mathsf{Setup}(\lambda)$
2: $(\mathsf{sk},\mathsf{pk}) \leftarrow \Pi.KeyGen(\lambda)$
3: $\{m_i\}_{i=1}^n \leftarrow\!\$\ \mathbb{M}$
4: $\{c_i\}_{i=1}^n \leftarrow \mathsf{Enc}(\mathsf{pk},m_i)$
5: $\{m_i^*\}_{i=1}^{n+1} \leftarrow \mathcal{A}(pp,\mathsf{pk},\{c_i\}_{i=1}^n)^{\mathcal{O}_{\mathsf{Dec}}(\cdot)}$
6: **return** $\bigwedge_{i=1}^{n+1} m_i = m_i^*$

Fig. 14: One-More-Decryption experiment

$\mathsf{Setup}(\lambda)$ *generates a group description* $\mathbb{G}$ *of order* q *with generator* g *and sets these to be pp.*
$\mathsf{KeyGen}(\lambda)$ *samples* $x \leftarrow\$ \mathbb{Z}_q$ *and sets* $(\mathsf{sk},\mathsf{pk}) = (x, g^x)$.
$\mathsf{Enc}(\mathsf{pk} = h, m)$ *samples* $y \leftarrow\$ \mathbb{Z}_q$ *and sets* $c = (c_1, c_2) = (g^y, m \cdot h^y)$.
$\mathsf{Dec}(\mathsf{sk} = x, c)$ *first computes* $s^{-1} = c_1^x$ *and outputs* $m = c_2 \cdot s^{-1}$.

C. Message Authentication Codes

A message authentication code consists of three algorithms $(\mathsf{KeyGen}, \mathsf{MAC}, \mathsf{Verify})$ defined below:

$\mathsf{KeyGen}(\lambda)$ generates keys K.
$\mathsf{MAC}(K, m)$ outputs tag μ for key K and message m.
$\mathsf{Verify}(K, m, \mu)$ outputs 1 if μ is valid for m under K, otherwise it returns 0.

MAC satisfies its correctness property if and only if

$$\forall m, K \leftarrow \mathsf{KeyGen}(\lambda), \mathsf{Verify}(K, m, \mathsf{MAC}(K, m)) = 1$$

Definition 11. *MAC is unforgeable if the advantage* $\mathsf{Adv}^{\mathsf{unforge}}_{\mathcal{A},\mathsf{MAC}}(\lambda)$ *is negligible in* λ *for a PPT adversary* $\mathcal{A}$ *to find, without* K*, a valid tag* μ^* *for a new message* m^**.* $\mathcal{A}$ *is given access to oracle* $\mathcal{O}_{\mathsf{MAC}}(\cdot)$*, which on input message* $m \neq m^*$ *returns the result of* $\mathsf{MAC}(K, m)$.

D. Commitment Schemes

A commitment scheme consists of three algorithms $(\mathsf{KeyGen}, \mathsf{Com}, \mathsf{Open})$ defined below:

$\mathsf{KeyGen}(\lambda)$ generates a commitment key ck.
$\mathsf{Com}(ck, m)$ outputs commitment c to message m, and its opening d.
$\mathsf{Open}(m, c, d)$ outputs 1 if c is valid for commitment for m with opening d, otherwise it returns 0.

A commitment satisfies its correctness property if and only if

$$\forall m, ck \leftarrow \mathsf{KeyGen}(\lambda), \mathsf{Open}(m, \mathsf{Com}(ck, m)) = 1$$

Definition 12. *A commitment scheme is binding if,* $\forall ck \leftarrow \mathsf{KeyGen}(\lambda)$*, then for any PPT adversary* $\mathcal{A}$ *that outputs* (m_0, m_1, d_0, d_1)*, (with* $m_0 \neq m_1$*) we have*

$$\mathsf{Adv}^{\mathsf{binding}}_{\mathcal{A},\mathsf{Com}}(\lambda) :=$$
$$\Pr[\mathsf{Open}(m_0, c, d_0) = 1 = \mathsf{Open}(m_1, c, d_1)] \leqslant \varepsilon(\lambda)$$

Definition 13. *A commitment scheme is hiding if, for all adversarially chosen* m_0, m_1*, then we have*

$$\mathsf{Adv}^{\mathsf{hiding}}_{\mathcal{A},\mathsf{Com}}(\lambda) := \Pr\left[b' \leftarrow \mathcal{A}(\mathsf{Com}(ck, m_b)) : b' \stackrel{?}{=} b\right] \leqslant \varepsilon(\lambda)$$

We now give proofs for Privacy Pass against our game-based definitions. All proofs in the section are recalled from Davidson et al. [6] with minor changes. More precisely, the proof for one-more token security is from the original work, we have slightly modified the authors' argument for unlinkability to fit our definition, and we have formally given the argument outlined for single signing key.

Theorem 5. PP *is correct if* NIZK *and* MAC *are correct.*

Proof. Let $(T, r) \leftarrow\$ \mathsf{Blind}(\mathsf{GenToken}(t))$ where $t \leftarrow\$ \mathbb{Z}_q$, therefore $\widetilde{T} = T^r$. When TI receives $\widetilde{T}$ it first computes $\widetilde{T}^{\mathsf{sk}} = f_{\mathsf{sk}}(\widetilde{T}) \leftarrow \mathsf{Sign}(\widetilde{T}, \mathsf{sk})$. It then computes a DLEQ NIZK proof $\pi_{\mathsf{sk}} \leftarrow \mathsf{Prove}(\mathsf{sk}, c_{\mathsf{sk}}, f_{\mathsf{sk}}(\widetilde{T}))$ and sends $(\widetilde{T}^{\mathsf{sk}}, \pi_{\mathsf{sk}})$ back to U. Since π_{sk} is a valid DLEQ proof, then $1 \leftarrow \mathsf{Verify}(\pi_{\mathsf{sk}}, c_{\mathsf{sk}}, \widetilde{T}, \widetilde{T}^{\mathsf{sk}})$. Therefore, the user computes $T^{\mathsf{sk}} \leftarrow \mathsf{Unblind}(\widetilde{T}^{\mathsf{sk}}, r)$ and the signing phase is complete. For the redemption phase, U computes a shared key $K \leftarrow H_2(t, T^{\mathsf{sk}})$ and sends $(t, \mathsf{MAC}_K(R))$ to TI. Given t, TI calculates $T^{\mathsf{sk}} = \mathsf{Sign}(\mathsf{GenToken}(t))$ and computes $\mathsf{MAC}_{H_2(t,T^{\mathsf{sk}})}(R)$. Since $K = H_2(t, T^{\mathsf{sk}})$ by definition, we conclude that $\mathsf{MAC}_K(R) = \mathsf{MAC}_{H_2(t,T^{\mathsf{sk}})}(R)$. □

Theorem 6. PP *is unlinkable.*

Proof. Let $\mathsf{Game}^b_0 = \mathsf{Exp}^{\mathsf{unlink}\text{-}b}_{\mathcal{A},\mathsf{PP}}(\lambda)$.
Now let Game_1 to be defined as Game^b_0 except with the change that we fix $b = 0$, i.e. $\mathsf{Game}_1 = \mathsf{Game}^0_0$
In PP, $\widetilde{T} = T^r$ for a randomly selected $r \leftarrow\$ \mathbb{Z}_q$. Since T is a generator of $\mathbb{G}$, $\widetilde{T}$ is a uniform element of $\mathbb{G}$, and therefore contains no information about T or t. Thus blind_b is distributed equally for both $b = 0$ and $b = 1$. Furthermore, the MAC tag s_1 depends on R, which is fixed by the game, and the key k_1, which is directly derived from t. Thus, since the view from TI of $\mathsf{U}\text{-}\mathsf{Issue}_1(t)$ is dependent only on $\widetilde{T}$ and the view from TI of $\mathsf{U}\text{-}\mathsf{Redeem}(t, f_{\mathsf{sk}}(T)))$ is dependent on t and T, we have the advantage of an adversary distinguishing these games is negligible in λ.

$$|\Pr[\mathsf{Game}_0 = 1] - \Pr[\mathsf{Game}_1 = 1]| \leqslant \varepsilon(\lambda).$$

Now the advantage of the adversary $\mathcal{A}$ against Game_1 is 0, since it is independent of the bit b.

$$\Pr[\mathsf{Game}_1 = 1] = 0$$

By the sequence of games Game_0 to Game_1, we have shown the advantage of an adversary distinguishing $\mathsf{Exp}^{\mathsf{unlink}\text{-}0}_{\mathcal{A},\mathsf{PP}^+}(\lambda)$ from $\mathsf{Exp}^{\mathsf{unlink}\text{-}1}_{\mathcal{A},\Pi}(\lambda)$ is negligble, that is:

$$\mathsf{Adv}^{\mathsf{unlink}}_{\mathcal{A},\mathsf{PP}}(\lambda) \leqslant \varepsilon(\lambda)$$

□

Theorem 7. PP *has one-more-Token security in the Random Oracle Model if El Gamal is secure against one-more-decryption attacks and* NIZK *is zero-knowledge.*

Proof. We begin by assuming the zero-knowledge property of the NIZK proof system, and thus the TI key is not leaked through the proof. Formally, this would be argued by swapping the prove algorithm with a simulator, which is independent of the secret key sk. If the adversary can detect this swap, then it has broken the zero-knowledge property of the NIZK system. This is omitted for conciseness. We now analyse an adversary that wins the one-more-token experiment.

Given an adversary $\mathcal{A}$ who can get n tokens signed by the TI, and then redeem $n+1$, we will produce an adversary $\mathcal{B}$ who wins the one-more-decryption game against El Gamal.

First, we consider the case it forges the Token Issuer token. $\mathcal{B}$ starts the game by being given $\mathbb{G}, q, g, Y$, and $\{(C_i, D_i)\}_{i \in [n+1]}$. $\mathcal{B}$ initiates $\mathcal{A}$, using $(\mathbb{G}, q, g)$ as the group description and Y as the commitment to the signing key, i.e. $Y = g^{\mathsf{sk}}$. $\mathcal{B}$ also selects a random permutation τ on $\mathbb{Z}_q$, but does not reveal it to $\mathcal{A}$. $\mathcal{A}$ can make two kinds of queries: polynomially many random oracle queries to H_1, H_2, and H_3, and at most n token signing queries.
When $\mathcal{A}$ makes a token signing query with blinded token $\widetilde{T}_i$, $\mathcal{B}$ selects a random $F_i \leftarrow\$ \mathbb{G}$, and sends the ciphertext $(\widetilde{T}_i, F_i)$ to the decryption oracle, which returns $M_i = F_i / \widetilde{T}_i^k$. $\mathcal{B}$ then returns

$F_i/M_i = \widetilde{T}_i^k$ to $\mathcal{A}$. It provides a forged DLEQ proof, which it can do because it can program the responses of H_3. Furthermore, the TI token can be simulated during this phase due to the uniformity of $f_{\mathsf{sk}}(T)$ in $\mathbb{G}$.

When $\mathcal{A}$ makes a random oracle request for $H_1(t)$, $\mathcal{B}$ replies with $\prod_{j\in[n+1]} C_j^{(a^{j-1})}$, where $a = \tau(t)$. Only with negligible probability $1/q$ will this value be the identity element of $\mathbb{G}$, in which case $\mathcal{B}$ aborts with failure.

When $\mathcal{A}$ makes a random oracle request for $H_2(t,B)$, $\mathcal{B}$ stores (t,B,K) in a table for a randomly selected K, and replies with K. If the same $H_2(t,B)$ is queried again, the same K will be returned. When $\mathcal{A}$ makes a random oracle request for H_3, $\mathcal{B}$ programs it to forge the DLEQ proof, as above.

At the end of the game, $\mathcal{A}$ will redeem $n+1$ tokens $\{(t_i,R_i,\mathsf{MAC}_{H_2(t_i,B_i)}(R_i))\}_{i\in[n+1]}$. $\mathcal{B}$ uses the H_3 table to look up the value of B_i used to generate the MAC key in each token, and it will be the case that $B_i = H_1(t_i)^k = \prod_{j\in[n+1]}(C_j^k)^{(\tau(t_i)^{j-1})}$ for each $i\in[n+1]$. If V is the (Vandermonde, and thus invertible) matrix with $V_{ij} = \tau(t_i)^{j-1}$, and U is the inverse of V, then it will be the case that $\prod_{j\in[n+1]} B^{U_{ij}} = C_i^k$ for each $i\in[n+1]$. $\mathcal{B}$ then outputs $M_i = D_i/C_i^k$ and wins the game.

In summary, we have the following bound on the advantage of $\mathcal{A}$:

$$\mathsf{Adv}^{\mathsf{one\text{-}more}}_{\mathcal{A},\mathsf{PP}}(\lambda) \leqslant \mathsf{Adv}^{\mathsf{one\text{-}more\text{-}dec}}_{\mathcal{B},\mathsf{Enc}}(\lambda)$$

□

Theorem 8. PP *has the single signing key property if the zero-knowledge proof system* NIZK *is sound and the commitment scheme* Com *has the binding property.*

Proof. Let $\mathsf{Game}_0 = \mathsf{Exp}^{\mathsf{single\text{-}key}}_{\mathcal{A},\mathsf{PP}}(\lambda)$.
The advantage of the adversary $\mathcal{A}$ against Game_0 is bound by an adversary against the soundness of NIZK. In particular, to construct the adversary $\mathcal{C}$ from $\mathcal{A}$, let $\mathcal{C}$ setup the game as described and wait for $\mathcal{A}$ to output the pair $(\mathsf{sk},\mathsf{sbt})$. It parses sbt as $(f_{\mathsf{sk}}(\widetilde{T}), c_{\mathsf{sk}}, \pi_{\mathsf{sk}})$, and forwards the tuple $(c_{\mathsf{sk}}, \pi_{\mathsf{sk}})$ to its soundness game. We see that $\mathcal{C}$ wins its experiment if $\mathcal{A}$ wins, since this only happens if it is able to produce a proof π_{sk} that (incorrectly) verifies sk as the key used in the creation of T. Note that this step implicitly assumes the binding property of the commitment scheme. If this is the case, then $(\mathsf{sk},\pi_{\mathsf{sk}}) \notin L_{R^*}$ and hence $\mathcal{C}$ wins. Thus, we conclude:

$$\mathsf{Adv}^{\mathsf{single\text{-}key}}_{\mathcal{A},\mathsf{PP}}(\lambda) = \Pr[\mathsf{Game}_0 = 1] \leqslant \mathsf{Adv}^{\mathsf{binding}}_{\mathcal{B},\mathsf{Com}}(\lambda) + \mathsf{Adv}^{\mathsf{sound}}_{\mathcal{C},\mathsf{NIZK}}(\lambda)$$

□

E. Correctness and Security Properties of Privacy Pass Plus

In this section, we formalise the game-based security experiments of Privacy Pass Plus. Intuitively, are modelled off the definitions presented in Section II-B with additional oracles to model stronger adversarial behaviour.

1) Oracles: The oracles used in this definition are identical, with the exception of an additional oracle $\mathcal{O}_{\mathsf{TI-Corrupt}}$. We recall all oracles for completeness.

$\mathcal{O}_{\mathsf{U\text{-}Issue}_1}$ This oracle does not require an input and initiates the issuance of tokens. It samples a seed t and runs the algorithm GenToken, then outputs blinded tokens $\widetilde{T}$. It adds the tuple $(\mathsf{sid},t,r,\widetilde{T})$ to the list IssuedTokens, where sid is a session ID incremented from 0, and r is the blinding randomness.

$\mathcal{O}_{\mathsf{TI\text{-}Issue}}$ For a secret key sk fixed by the experiment, this oracle takes as input $\widetilde{T}$ and returns signed blinded tokens $f_{\mathsf{sk}}(\widetilde{T})$.

$\mathcal{O}_{\mathsf{U\text{-}Issue}_2}$ This oracle takes as input $(\mathsf{sid}, f_{\mathsf{sk}}(\widetilde{T}))$, unblinds $f_{\mathsf{sk}}(\widetilde{T})$ and returns the signed tokens $f_{\mathsf{sk}}(T)$ and the corresponding seed t (matched from the list IssuedTokens on sid). If $(\mathsf{sid},\cdot,\cdot,\cdot) \notin$ IssuedTokens then it returns $\perp$.

$\mathcal{O}_{\mathsf{U\text{-}Redeem}}$ This takes as input the signed tokens and session ID $(\mathsf{sid}, f_{\mathsf{sk}}(T))$. It locates $(\mathsf{sid},\cdot,\cdot,\cdot) \in$ IssuedTokens and extracts the seed t and computes verification data s. It checks a sidList list for sid for every token redeemed to ensure it has not previously been spent. If sid is already in sidList, it aborts, else it outputs (t,s) and adds sid to the sidList to mark the token as spent.

$\mathcal{O}_{\mathsf{TI\text{-}Redeem}}$ This oracle takes as input the seed t and computes verification data s. It checks if s is valid with respect to t. Further, it checks if $t \notin$ SpentTokens, in which case it adds t and returns 1, else it returns 0.

$\mathcal{O}_{\mathsf{Chal}}$ This challenge oracle is used in the unlinkability experiment. It takes input two signed blinded tokens $f_{\mathsf{sk}}(\widetilde{T}_0)$ and $f_{\mathsf{sk}}(\widetilde{T}_1)$ (and resp. sid), and then executes and returns $\mathcal{O}_{\mathsf{U\text{-}Redeem}}(\mathsf{sid}_b, f_{\mathsf{sk}}(T_b))$ for both $b=0$ and $b=1$. If the oracle returns $\perp$ for either $b=0$ or $b=1$, then it simply returns $\perp$. Else it returns (t_b,s_b), according to the bit b defined by the experiment.

$\mathcal{O}_{\mathsf{TI-Corrupt}}$ This oracle reveals the signing key of the TI. Specifically, it returns sk and updates a variable CorruptTI (initially set to *false*) to *true*. It also sets the check-bit $\mathsf{cb}=1$.

Correctness. Correctness property ensures that tokens generated by both the TI and server are valid, provided the algorithms are correctly followed. We define the correcness in Definition 1.

Definition 14. *We say that the protocol* Π *is correct if the probability that a token generated in the* Π.IssuePhase *can successfully be redeemed in the* Π.RedeemPhase *is negligibly close to 1 for* $\mathsf{cb}\in\{0,1\}$ *i.e.*

$$\Pr[1 \leftarrow \Pi.\mathsf{RedeemPhase}(t, f_{sk}(T), f_{\mathsf{rk}}(f_{sk}(T),\mathsf{cb})] > 1-\varepsilon(\lambda)$$

Given

$$t \xleftarrow{\$} \mathbb{Z}_q,$$
$$\mathsf{cb} \xleftarrow{\$} \{0,1\},$$
$$f_{sk}(T), f_{\mathsf{rk}}(f_{sk}(T)) \leftarrow \Pi.\mathsf{IssuePhase}(t)$$

Unlinkability$^+$. The unlinkability definition is adopted directly from Definition 3. This is because the adversary for unlinkability, and so it is not needed to provide it with the new oracle $\mathcal{O}_{\mathsf{TI-Corrupt}}$, however, we cannot use the definition directly as the modelling of the algorithms have changed and thus the experiment implicitly has, too. The definition is formally stated in Definition 15.

Definition 15. *We say that the protocol is unlinkable*$^+$ *if the advantage of the PPT adversary* $\mathcal{A}$ *in distinguishing the experiment* $\mathsf{Exp}^{\mathsf{unlink}^+\text{-}0}_{\mathcal{A},\Pi}(\lambda)$ *from* $\mathsf{Exp}^{\mathsf{unlink}^+\text{-}1}_{\mathcal{A},\Pi}(\lambda)$ *defined in Figure 15 is negligibly small (in* λ*), i.e.*

$$\mathsf{Adv}^{\mathsf{unlink}^+}_{\mathcal{A},\Pi}(\lambda) := \left|\Pr\left[\mathsf{Exp}^{\mathsf{unlink}^+\text{-}0}_{\mathcal{A},\Pi}(\lambda)=1\right] - \Pr\left[\mathsf{Exp}^{\mathsf{unlink}^+\text{-}1}_{\mathcal{A},\Pi}(\lambda)=1\right]\right| \leqslant \varepsilon(\lambda)$$

One-More-Token$^+$ *Security.* Based on the game presented in Figure 4, the adversary is tasked to produce $n+1$ valid tokens

$\mathsf{Exp}^{\mathsf{unlink}^+\text{-}b}_{\mathcal{A},\Pi}(\lambda)$

1: $pp \leftarrow \mathsf{Setup}(\lambda)$
2: $\mathsf{bt}_0 \leftarrow \mathsf{U\text{-}Issue}_1()$
3: $\mathsf{bt}_1 \leftarrow \mathsf{U\text{-}Issue}_1()$
4: $(\mathsf{sbt}_0, \mathsf{sbt}_1) \leftarrow \mathcal{A}_0(\mathsf{bt}_0, \mathsf{bt}_1)$
5: $\mathsf{tok}_0 \leftarrow \mathsf{U\text{-}Issue}_2(\mathsf{sbt}_0)$
6: $\mathsf{tok}_1 \leftarrow \mathsf{U\text{-}Issue}_2(\mathsf{sbt}_1)$
7: **abort if** $\mathsf{tok}_0 = \bot \vee \mathsf{tok}_1 = \bot$
8: $b' \leftarrow \mathcal{A}_1^{\mathcal{O}_{\mathsf{U-Redeem}}, \mathcal{O}_{\mathsf{U-Issue}_1}, \mathcal{O}_{\mathsf{U-Issue}_2}, \mathcal{O}_{\mathsf{Chal}}}$
9: **return** $b' \stackrel{?}{=} b$

Fig. 15: Unlinkability experiment

after at most n token issuance oracle queries. The main difference is that the adversary gets access to the $\mathcal{O}_{\mathsf{TI-Corrupt}}$ oracle, which reveals the TI's secret key. Note that in a departure from what can be considered the norm, the adversary is allowed to query this oracle even on challenge TI, however this sets the check-bit $\mathsf{cb} = 1$ to capture the case the server observes unexpected behaviour. We formally state the experiment in Definition 16.

Definition 16. *We say that the protocol has one-more-token+ security if the advantage of the PPT adversary $\mathcal{A}$ in the experiment $\mathsf{Exp}^{\mathsf{one-more}^+}_{\mathcal{A},\Pi}(\lambda)$ in Figure 16 is negligible in the security parameter λ, i.e.*

$$\mathsf{Adv}^{\mathsf{one-more}^+}_{\mathcal{A},\Pi}(\lambda) := \left|\Pr\left[\mathsf{Exp}^{\mathsf{one-more}^+}_{\mathcal{A},\Pi}(\lambda) = 1\right]\right| \leqslant \varepsilon(\lambda)$$

Single Signing Key$^+$. Much like unlinkability, the adversary in this case is the TI and so does not benefit from the additional oracle, hence the definition is intuitively the same as in Definition 4, with trivial changes to account for differences in the algorithm modelling.

Definition 17. *We say that the protocol has the single signing key$^+$ property if the advantage of the PPT adversary $\mathcal{A}$ in the experiment $\mathsf{Exp}^{\mathsf{single-key}^+}_{\mathcal{A},\Pi}(\lambda)$ defined in Figure 17 is negligible in λ, i.e.*

$$\mathsf{Adv}^{\mathsf{single-key}^+}_{\mathcal{A},\Pi}(\lambda) := \left|\Pr\left[\mathsf{Exp}^{\mathsf{single-key}^+}_{\mathcal{A},\Pi}(\lambda) = 1\right]\right| \leqslant \varepsilon(\lambda)$$

Theorem 2. PP^+ enjoys the unlinkable$^+$ property.

Proof. Let $\mathsf{Game}^b_0 = \mathsf{Exp}^{\mathsf{unlink}^+\text{-}b}_{\mathcal{A},\mathsf{PP}^+}(\lambda)$.
Now let Game_1 to be defined as Game^b_0 except with the change that we fix $b = 0$, i.e. $\mathsf{Game}_1 = \mathsf{Game}^0_0$

$\mathsf{Exp}^{\mathsf{one-more}^+}_{\mathcal{A},\Pi}(\lambda)$

1: $pp \leftarrow \mathsf{Setup}(\lambda)$
2: $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KeyGen}(\lambda)$
3: $\{\mathsf{tok}_i\}_{i=1}^{n+1} \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{TI-Redeem}}, \mathcal{O}_{\mathsf{TI-Issue}}, \mathcal{O}_{\mathsf{TI-Corrupt}}}$
4: **for** $i \in [1, n+1]$
5: $\quad res_i \leftarrow \mathsf{TI\text{-}Redeem}(\mathsf{sk}, \mathsf{U\text{-}Redeem}(\mathsf{tok}_i))$
6: **return** $\bigwedge_{i=1}^{n+1} res_i$

Fig. 16: One-more-token+ experiment

$\mathsf{Exp}^{\mathsf{single-key}^+}_{\mathcal{A},\Pi}(\lambda)$

1: $pp \leftarrow \mathsf{Setup}(\lambda)$
2: $\mathsf{bt} \leftarrow \mathsf{U\text{-}Issue}_1()$
3: $(\mathsf{sbt}, \mathsf{sk}, \mathsf{pk}) \leftarrow \mathcal{A}(\mathsf{bt})^{\mathcal{O}_{\mathsf{U-Issue}_1}, \mathcal{O}_{\mathsf{U-Issue}_2}}$
4: $\mathsf{tok}^* \leftarrow \mathsf{U\text{-}Issue}_2(\mathsf{pre}, \mathsf{sbt}, \mathsf{pk})$
5: **if** $\mathsf{tok}^* = \bot$, **abort**
6: $\mathsf{tok} \leftarrow \mathsf{TI\text{-}Issue}(\mathsf{sk}, \mathsf{pre})$
7: $b \leftarrow (\mathsf{tok}^* \stackrel{?}{=} \mathsf{tok})$
8: **return** $\bar{b}$

Fig. 17: Single Signing Key$^+$ experiment

In PP^+, $\widetilde{T} = T^r$ for a randomly selected $r \leftarrow\$ \mathbb{Z}_q$. Since T is a generator of $\mathbb{G}$, $\widetilde{T}$ is a uniform element of $\mathbb{G}$, and therefore contains no information about T or t. Thus blind_b is distributed equally for both $b = 0$ and $b = 1$. Furthermore, the MAC tags s_1, s_2 depend on R_{TI}, R_S, respectively, which are fixed by the game. We also note that the keys k_1 and k_2 are directly derived from t. Thus, since the view from TI in the $\mathsf{U\text{-}Issue}_1(t)$ phase is dependent only on $\widetilde{T}$ and the view from TI in $\mathsf{U\text{-}Redeem}(t, f_{\mathsf{sk}}(T), f_{\mathsf{rk}}(f_{\mathsf{sk}}(T)))$ is dependent on t and T, and $\widetilde{T}$ is independent of (t, T), we have the advantage of an adversary distinguishing these games is negligible in λ.

$$|\Pr[\mathsf{Game}_0 = 1] - \Pr[\mathsf{Game}_1 = 1]| \leqslant \varepsilon(\lambda).$$

Now the advantage of the adversary $\mathcal{A}$ against Game_1 is 0, since it is independent of the bit b.

$$\Pr[\mathsf{Game}_1 = 1] = 0$$

By the sequence of games Game_0 to Game_1, we have shown the advantage of an adversary distinguishing $\mathsf{Exp}^{\mathsf{unlink\text{-}0}}_{\mathcal{A},\mathsf{PP}^+}(\lambda)$ from $\mathsf{Exp}^{\mathsf{unlink\text{-}1}}_{\mathcal{A},\Pi}(\lambda)$ is negligble, that is:

$$\mathsf{Adv}^{\mathsf{unlink}^+}_{\mathcal{A},\mathsf{PP}^+}(\lambda) \leqslant \varepsilon(\lambda).$$

□

Theorem 3. PP^+ has one-more-token+ security in the Random Oracle Model if El Gamal is secure against one-more-decryption attacks and NIZK is zero-knowledge.

Proof. We begin by assuming the zero-knowledge property of the NIZK proof system, and thus the TI key is not leaked through the proof. Formally, this would be argued by swapping the prove algorithm with a simulator, which is independent of the secret key sk. If the adversary can detect this swap, then it has broken the zero-knowledge property of the NIZK system. This is omitted for conciseness. We now analyse an adversary that wins the one-more-token experiment. Given an adversary $\mathcal{A}$ who can get n tokens signed by the Server, and then redeem $n+1$, we will produce an adversary $\mathcal{B}$ who wins the one-more-decryption game against El Gamal. Intuitively, we will do this twice, for tokens signed by Server and TI, respectively. Without loss of generality, we assume the Server check-bit $\mathsf{cb} = 1$.

First, we consider the case it forges the Token Issuer token, and therefore has not queried the $\mathcal{O}_{\mathsf{TI-Corrupt}}$ oracle. $\mathcal{B}$ starts the game by being given $\mathbb{G}, q, g, Y$, and $\{(C_i, D_i)\}_{i \in [n+1]}$ are the ElGamal ciphertexts. More precisely, $C_i = X^{r_i}$ and $D_i = Y^{r_i} \cdot M_i$.

$\mathcal{B}$ initiates $\mathcal{A}$, using $(\mathbb{G},q,g)$ as the group description and Y as the commitment to the signing key , i.e. $Y = g^{\mathsf{sk}}$.. $\mathcal{B}$ also selects a random permutation τ on $\mathbb{Z}_q$, but does not reveal it to $\mathcal{A}$. $\mathcal{A}$ can make two kinds of queries: polynomially many random oracle queries to H_1, H_2, and H_3, and at most n token signing queries.

When $\mathcal{A}$ makes a token signing query with blinded token $\widetilde{T}_i$, $\mathcal{B}$ selects a random $F_i \leftarrow\!\$\, \mathbb{G}$, and sends the ciphertext $(\widetilde{T}_i,F_i)$ to the decryption oracle, which returns $M_i = F_i/\widetilde{T}_i^k$. the adversary $\mathcal{B}$ then returns $F_i/M_i = \widetilde{T}_i^k$ to $\mathcal{A}$. It provides a forged DLEQ proof, which it can do because it can program the responses of H_3. Furthermore, the Server token can be simulated for during this phase to the uniformity of $f_{\mathsf{rk}}(f_{\mathsf{sk}}(T))$ in $\mathbb{G}$. The challenger replaces $f_{\mathsf{rk}}(f_{\mathsf{sk}}(\widetilde{T}_i)))$ with randomly sampled group elements, this change is undetectable by $\mathcal{B}$ because rk is uniformly sampled from $\mathbb{Z}_q$ and thus the distribution of $f_{\mathsf{rk}}(f_{\mathsf{sk}}(\widetilde{T}_i)))$ is both uniform in $\mathbb{G}$ and independent of $\widetilde{T}_i$.

When $\mathcal{A}$ makes a random oracle request for $H_1(t)$, $\mathcal{B}$ replies with $\prod_{j\in[n+1]} C_j^{(a^{j-1})}$, where $a = \tau(t)$. Only with negligible probability $1/q$ will this value be the identity element of $\mathbb{G}$, in which case $\mathcal{B}$ aborts with failure.

When $\mathcal{A}$ makes a random oracle request for $H_2(t,B)$, $\mathcal{B}$ stores (t,B,K) in a table for a randomly selected K, and replies with K. If the same $H_2(t,B)$ is queried again, the same K will be returned. When $\mathcal{A}$ makes a random oracle request for H_3, $\mathcal{B}$ programs it to forge the DLEQ proof, as above.

At the end of the game, $\mathcal{A}$ will redeem $n+1$ tokens $\{(t_i,R_i,\mathsf{MAC}_{H_2(t_i,B_i)}(R_i))\}_{i\in[n+1]}$. $\mathcal{B}$ uses the H_3 table to look up the value of B_i used to generate the MAC key in each token, and it will be the case that $B_i = H_1(t_i)^k = \prod_{j\in[n+1]}(C_j^k)^{(\tau(t_i)^{j-1})}$ for each $i\in[n+1]$. If V is the (Vandermonde, and thus invertible) matrix with $V_{ij} = \tau(t_i)^{j-1}$, and U is the inverse of V, then it will be the case that $\prod_{j\in[n+1]} B^{U_{ij}} = C_i^k$ for each $i \in [n+1]$. The adversary $\mathcal{B}$ then outputs $M_i = D_i/C_i^k$ and wins the game.

Now we consider the case the adversary has made a query to the $\mathcal{O}_{\mathsf{TI-Corrupt}}$ oracle. This case the check-bit $\mathsf{cb} = 1$ and so the adversary must create a valid Server Token. The argument is similar, so we omit the details. However we note again that when embedding this challenge, the TI tokens can be simulated due to the uniform distribution of tokens in $\mathbb{G}$, in particular, it is not detectable by the adversary that the challenge tokens are not 'signed' by TI with sk, since T^r has the same distribution as $(T^{\mathsf{sk}})^r$. Furthermore, there is no DLEQ proof on the Server, so we do not need to make use of the random oracle H_3.

In summary, we have the following bound on the advantage of $\mathcal{A}$:

$$\mathsf{Adv}^{\mathsf{one\text{-}more}^+}_{\mathcal{A},\mathsf{PP}^+}(\lambda) \leqslant 2\cdot\mathsf{Adv}^{\mathsf{one\text{-}more\text{-}dec}}_{\mathcal{B},\mathsf{Enc}}(\lambda)$$

□

Theorem 4. PP^+ has the single-signing-key$^+$ property if the zero-knowledge proof system NIZK is sound and the commitment scheme Com has the binding property.

Proof. Let $\mathsf{Game}_0 = \mathsf{Exp}^{\mathsf{single\text{-}key}^+}_{\mathcal{A},\mathsf{PP}^+}(\lambda)$.
The advantage of the adversary $\mathcal{A}$ against Game_0 is bound by an adversary against the soundness of NIZK. In particular, to construct the adversary $\mathcal{C}$ from $\mathcal{A}$, let $\mathcal{C}$ setup the game as described and wait for $\mathcal{A}$ to output the pair $(\mathsf{sk},\mathsf{sbt})$. It parses sbt as $(f_{\mathsf{sk}}(\widetilde{T}),f_{\mathsf{rk}}(f_{\mathsf{sk}}(\widetilde{T})), c_{\mathsf{sk}}, \pi_{\mathsf{sk}})$, and forwards the tuple $(c_{\mathsf{sk}}, \pi_{\mathsf{sk}})$ to its soundness game. We see that $\mathcal{C}$ wins its experiment if $\mathcal{A}$ wins, since this only happens if it is able to produce a proof π_{sk} that (incorrectly) verifies sk as the key used in the creation of T. Note that this step implicitly assumes the binding property of the commitment scheme. If this is the case, then $(\mathsf{sk},\pi_{\mathsf{sk}}) \notin L_{R^*}$) and hence $\mathcal{C}$ wins. Thus, we conclude:

$$\mathsf{Adv}^{\mathsf{single\text{-}key}^+}_{\mathcal{A},\mathsf{PP}^+}(\lambda) = \Pr[\mathsf{Game}_0 = 1] \leqslant \mathsf{Adv}^{\mathsf{binding}}_{\mathcal{B},\mathsf{Com}}(\lambda) + \mathsf{Adv}^{\mathsf{sound}}_{\mathcal{C},\mathsf{NIZK}}(\lambda)$$

□